

Appunti di Sistemi Distribuiti

Nicholas Scorrano

Anno Accademico 2025/2026

Indice

Capitolo 4

Struttura del Web

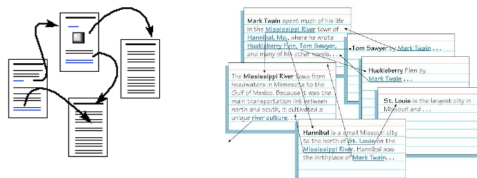
4.1 Ipertesti

Il fondamento logico per l'organizzazione delle informazioni sul Web è l'ipertesto (hypertext).

Un ipertesto è un insieme di contenuti (testuali e non) che si caratterizza per la sua non linearità.

Le informazioni, infatti, non vengono lette in modo sequenziale, ma vi si accede seguendo percorsi liberi definiti da collegamenti ipertestuali, i cosiddetti link.

A livello architetturale l'insieme degli ipertesti è rappresentabile come una rete con una struttura a grafo orientato.



4.2 Pagine Web e file HTML

L'ipertesto prende forma concreta nella pagina Web, la quale è composta da diverse risorse (oggetti) di cui il browser deve fare il rendering, come testo, immagini o musica.

La struttura portante e i contenuti della pagina, inclusi i riferimenti alle altre risorse, sono definiti all'interno di un file scritto in HTML (HyperText Markup Language), il linguaggio standard per la scrittura degli ipertesti.

Nel sistema, anche lo stesso file HTML è considerato a tutti gli effetti una risorsa

4.3 Meccanismi di Identificazione Univoca

4.3.1 URL - Uniform Resource Locator

Ogni singola risorsa presente sul Web deve essere identificata in modo univoco.

Lo strumento principale per farlo è 'URL, il cui compito non è solo **identificare**, ma anche localizzare fisicamente la risorsa, indicando in modo preciso dove andarla a trovare.

Struttura dell'URL

Un indirizzo URL si struttura unendo cinque componenti principali :

1. **Nome del protocollo**

Indica quale protocollo usare per il trasferimento dei dati

2. **Indirizzo IP**

L'indirizzo dell'host (o l'host name che andrà risolto tramite DNS)

3. **Porta di destinazione**

Parametro opzionale

4. **Percorso (cammino)**

Il percorso nell'host seguendo una gerarchia a cartelle

5. **Identificatore della risorsa**

Il nome del file finale

protocollo://indirizzo_IP/hostname[:porta]/cammino/risorsa
1. 2. 3. 4. 5.

Figura 4.1: Struttura di un URL

4.3.2 URN - Uniform Resource Name

Mentre l'URL indica dove si trova un oggetto, l'URN è un identificatore che definisce cosa è una risorsa, in modo del tutto indipendente dalla sua posizione fisica nella rete.

Questo garantisce due proprietà essenziali :

- **Persistenza**

Il nome assegnato tramite URN rimane valido anche se la risorsa viene fisicamente spostata su un altro server

- **Astrazione**

L'identificatore è slegato e non contiene alcuna informazione relativa al protocollo di accesso

Struttura dell'URN

La struttura di un URN segue il formato

1. **urn**

Identifica il tipo di identificatore (sempre fisso)

2. **<NID> - Namespace Identifier**

Identifica il tipo di risorsa

3. **<NSS> - Namespace Specific String**

Codice identificativo univoco della risorsa

```
urn:isbn:12345
urn:uuid:f81d4fae-7dec-11d0-a765-00a0c91e6bf6
```

Figura 4.2: esempi di assegnazione identificativi univoci

```
http://biblioteca.it/libri/urn:isbn:9788804668237
http://miosito.it/user/[urn:uuid:]f47ac10b-58cc-4372-a567-
0e02b2c3d479
```

Figura 4.3: esempio di identificazione di una risorsa in URL

4.4 I linguaggi del Web

All'interno dell'architettura del Web, i dati e i documenti testuali vengono espressi utilizzando una serie di linguaggi standardizzati, ognuno con uno scopo ben preciso.

4.4.1 Struttura, Stile e Dati

HTML - HyperText Markup Language

Il linguaggio fondamentale è l'**HTML**, il quale viene impiegato per definire la struttura portante e i contenuti effettivi della pagina Web.

CSS - Cascade Style Sheets

Per gestire l'**aspetto estetico** (rendering visivo, layout e formazione), all'interno delle pagine HTML può essere integrato del codice CSS.

XML e JSON

Quando invece l'obiettivo primario non è la visualizzazione ma la mera rappresentazione dei dati e il loro scambio tra applicazioni, si utilizzano linguaggi standard specifici.

I più importanti e diffusi in questo ambito sono l'**XML** e il **JSON**.

4.4.2 Dati Multimediali e Non Testuali

Oltre al testo, il Web è ricco di contenuti non testuali, come immagini, file audio e documenti video.

Poiché i protocolli come l'HTTP sono orientati al testo, si sfrutta la **codifica MIME**.

Questo meccanismo permette di trasformare file complessi e binari in una stringa di caratteri testuali convenzionali, rendendoli adatti a viaggiare sulla rete.

4.4.3 Linguaggi di scripting

Per fare in modo che una pagina Web non sia statica ma possa offrire funzionalità avanzate e arricchire l'interazione con l'utente, essa può contenere al suo interno del codice eseguibile.

Questo codice è tipicamente espresso in linguaggi di scripting, principalmente **Javascript**.

Capitolo 5

Protocollo HTTP - HyperText Transfer Protocol

L'HTTP è il protocollo di livello applicativo fondamentale per il Web.

5.1 Modello Client-Server

In questo ecosistema, le responsabilità sono divise in :

5.1.1 Client

È tipicamente rappresentato da un browser web.

Il suo compito è inviare richieste e occuparsi del rendering contenuti per l'utente, un processo che può adattarsi a dispositivi diversi, inclusi quelli "non visuali".

5.1.2 Server

È il Web server, che riceve le richieste e invia gli oggetti corrispondenti in risposta.

Un aspetto importante è che oggetti diversi che compongono una singola pagina Web possono essere prelevati anche da server Web differenti.

5.2 Fase di Connessione

L'HTTP non trasporta fisicamente i dati, ma si affida al protocollo di trasporto TCP.

La comunicazione avviene attraverso questa procedura :

1. Il client avvia una connessione TCP(creando una socket) verso l'indirizzo del server, tipicamente sulla porta 80.
2. Il server accetta la connessione.
3. Client e Server si scambiano i messaggi HTTP.
4. La connessione TCP viene chiusa.

5.3 Natura "Stateless"

Una delle scelte architetturali più rilevanti dell'HTTP è l'essere un protocollo **stateless**, ovvero privo di stato

5.3.1 Vantaggi

Il server non mantiene in memoria alcuna informazione sulle richieste precedenti del client.

Questo alleggerisce i server, evitando la complessità dei protocolli che mantengono lo stato.

5.3.2 Svantaggi

Ogni singola richiesta deve contenere tutte le informazioni necessarie per la sua esecuzione, aumentando l'overhead.

Per ovviare a questo limite e gestire sessioni HTTP si avvale dell'uso dei Cookie

5.4 Gestione delle Connessioni

La gestione delle connessioni TCP fa una grande differenza sulle prestazioni e si divide in 2 approcci principali

5.4.1 HTTP non persistente

Storicamente utilizzato in HTTP/1.0.

Questo modello prevede che su una connessione TCP appena aperta venga inviato al massimo un solo oggetto, dopodiché la connessione viene chiusa.

Se una pagina necessita di 10 immagini, il client dovrà aprire e chiudere connessioni ripetutamente.

Il tempo di risposta per scaricare ogni singolo oggetto è pari a **2 RTT (Round Trip Time) + il tempo di trasmissione dell'oggetto**

5.4.2 HTTP persistente

Adottato da HTTP/1.1, HTTP/2.0 e HTTP/3.0 questo modello lascia la connessione aperta.

Sfruttano un'unica connessione, il server può inviare al client più oggetti in sequenza.

In condizione in cui l'RTT è molto maggiore del tempo di trasmissione, questo approccio dimezza all'incirca i tempi di caricamento per ogni risorsa successiva alla prima.

5.5 Struttura e Formato dei Messaggi

HTTP è un protocollo orientato al carattere, di conseguenza, il formato dei suoi messaggi si basa su testo leggibile (ASCII).

5.5.1 Formato delle Richieste

Esempio messaggio di richiesta

```
GET /index.html HTTP/1.1\Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0)
Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
```

Caratteri speciali

Ogni messaggio di richiesta è strutturato in una sequenza precisa di righe separate tra loro tramite due speciali caratteri di controllo

- **cr-carriage return (ritorno a capo)**
- **lf-line feed (nuova linea)**

Riga di richiesta

È la riga di apertura del messaggio.

Contiene 3 parametri separati da uno spazio (sp)

- Il **metodo** HTTP che il client vuole eseguire
- L'**URL** identificativo della risorsa
- La **versione** del protocollo HTTP in uso

Esempio: Riga di richiesta

```
GET /index.html HTTP/1.1\r\n
```

Righe di intestazione

Subito dopo la riga di richiesta, si collocano gli header.

Sono strutturati nel formato **nome-campo: valore** e seguiti da un **cr lf**.

Servono al client per passare vari parametri al server, come ad esempio :

- **Host**
L'host specifico in cui risiede l'oggetto
- **User-Agent**
Il tipo di browser in uso

- **Accept**

Preferenze sulla negoziazione del contenuto tramite formati MIME

- **Preferenze aggiuntive** Come la lingua (Accept-Language) o il tipo di compressione accettata (Accept-Encoding)

Esempio Righe di intestazione

```
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0)
Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
```

5.5.2 Formato delle Risposte

I messaggi restituiti dal server iniziano con una riga di stato, seguita dalle righe di intestazione e infine dai dati effettivi (come il file HTML o l'immagine richiesta)

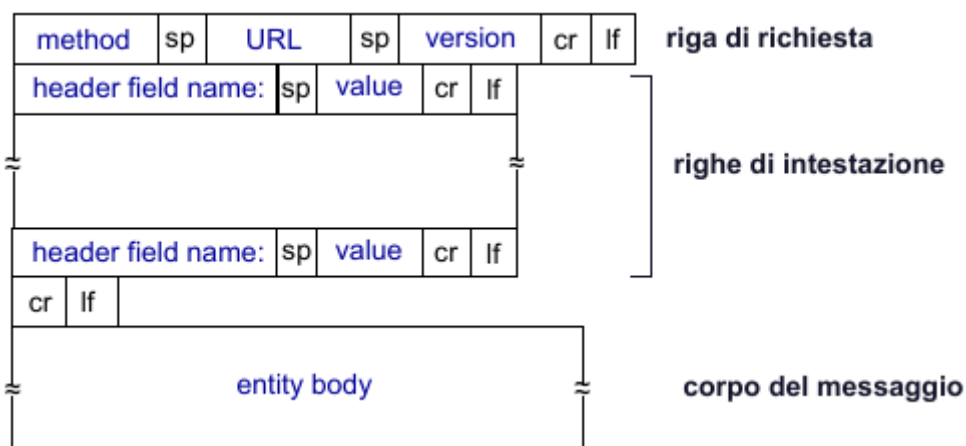


Figura 5.1: Struttura del messaggio generale

5.6 Metodi HTTP

I metodi HTTP definiscono l'azione specifica che il client desidera effettuare sulla risorsa indicata dall'URL.

Prima di elencarli è essenziale comprendere 2 parametri fondamentali con cui questi metodi vengono classificati

5.6.1 Le proprietà dei metodi

I metodi si distinguono in base a 2 proprietà che descrivono il loro impatto sui dati presenti nel server

Metodi Sicuri

Un metodo viene definito sicuro se la sua esecuzione non modifica in alcun modo lo stato del server.

Serve, in sostanza solo per la lettura

Metodi Idempotenti

Un metodo è considerato idempotente quando l'effetto che ha sul server dopo una singola richiesta andata a buon fine è esattamente identico all'effetto che si otterrebbe eseguendo più richieste identiche in modo consecutivo.

5.6.2 Metodo GET

Il metodo GET richiede al server di recuperare la risorsa specifica dall'URL o qualunque informazione a essa relativa.

È il metodo più comune per ottenere dati in vari formati, come documenti HTML, XML, JSON o contenuti multimediali.

Se la richiesta necessita di parametri, questi vengono agganciati direttamente in coda all'URL.

Proprietà

Trattandosi di un metodo di sola lettura, la GET è classificata come metodo **sicuro** e **idempotente**.

5.6.3 Metodo HEAD

La HEAD è praticamente identica alla GET, ma con una differenza cruciale : il server non deve restituire il corpo del messaggio nella risposta, ma solo le righe di intestazione.

L'uso tipico di questo metodo è il **debugging** oppure l'ottenimento di informazioni meta-dati su una risorsa

Proprietà

Esattamente come la GET, anche la HEAD è un metodo **sicuro** e **idempotente**.

Esempio di GET

```
GET myCompany/orders?item="Q2345"&date="2022/03" HTTP1.1
```

5.6.4 Metodo OPTIONS

Questo metodo rappresenta una richiesta per ottenere informazioni sulle opzioni di comunicazione permesse relativamente alla risorsa identificata dall'URL.

Proprietà

È un metodo **sicuro** e **idempotente**.

5.6.5 Metodo TRACE

Viene utilizzato per invocare un loopback del messaggio di richiesta a livello applicativo da parte del server, utile anch'esso per scopi diagnostici.

Proprietà

È un metodo **sicuro** e **idempotente**.

5.6.6 Metodo POST

La POST viene utilizzata per comunicare al server dei dati che devono essere elaborati, o per richiedere che il server accetti la risorsa inclusa nella richiesta come un nuovo "subordinato dall'URL indicato.

A differenza della GET, i dati non vengono passati nell'URL, ma sono inseriti come documento autonomo all'interno del corpo della richiesta, subito dopo le righe di intestazione.

L'uso tipico è trasportare i dati compilati da un utente all'interno di un form.

Proprietà

Poichè la POST serve tipicamente a creare o aggiornare dati sul server, **non è un metodo sicuro e non è idempotente**.

Esempio di POST

```
POST myCompany/orders HTTP1.1
Content-Length: 59
Content-Type: application/x-www-form-urlencoded
update=true&oldItem="Q2345"&newItem="Q68254"&date="2022/04"
```

5.6.7 Metodo PUT

La PUT richiede che la risorsa fornita venga memorizzata e caricata sul server all'URL specificato.

La sua particolarità è che, se nell'URL indicato esiste già un file, questo **viene completamente sostituito** dal nuovo contenuto presente nel corpo della richiesta.

Proprietà

Modificando i file, la PUT non è un metodo sicuro.

Tuttavia a differenza della POST, **è un metodo idempotente** : caricare o sovrascrivere lo stesso file una volta oppure cento volte porta sempre all'identico stato finale sul server

5.6.8 Metodo DELETE

Come deducibile dal nome, questo metodo richiede esplicitamente al server di **eliminare la risorsa** identificata dall'URL.

Proprietà

Non essendo un'operazione di sola lettura, non è un metodo sicuro.

Tuttavia, **è un metodo idempotente**: richiedere l'eliminazione di un file già eliminato non comporta ulteriori cambiamenti di stato nel server.

5.7 Codici di stato

I codici di stato sono valori numerici, accompagnati da una breve descrizione testuale, restituiti dal Web server al client all'interno dei messaggi di risposta HTTP.

La loro funzione è comunicare l'esito dell'operazione richiesta.

Fisicamente, si trovano nella riga di stato, ovvero la primissima riga del messaggio di risposta inviato dal server

5.7.1 Le 5 classi dei codici di stato

I codici sono suddivisi rigorosamente in 5 diverse classi, categorizzate in base al 1° numero della cifra.

1xx (Informational)

I codici che iniziano con 1 indicano una comunicazione di tipo informativo.

Segnalano al client che la richiesta è stata ricevuta e che il server sta continuando regolarmente la sua elaborazione.

2xx (Success)

I codici che iniziano con 2 vengono utilizzati per confermare un successo.

Indica che la richiesta del client è stata correttamente ricevuta, compresa, accettata ed elaborata dal server.

3xx (Redirection)

Questa classe di codici segnala che la risorsa non può essere fornita direttamente, ma che è necessario che il client intraprenda ulteriori azioni per poter completare la richiesta in modo corretto.

4xx (Client Error)

I codici 4xx indicano che il problema è imputabile a chi ha fatto la richiesta.

Segnalano che la richiesta inviata dal client presenta una sintassi errata oppure risulta incomprensibile al server.

5xx (Server Error)

I codici 5xx indicano un errore imputabile all'infrastruttura di destinazione.

Segnalano che il server non è riuscito a soddisfare una richiesta che risultava apparentemente valida.

5.8 I Cookie e il mantenimento dello stato

Per risolvere il limite imposto dall'assenza di stato, i client e i server HTTP si avvalgono dei cookie per riuscire a mantenere in memoria lo stato nelle transizioni.

Questa tecnologia viene impiegata tipicamente per 3 scopi principali:

1. Gestione dell'autorizzazione e autenticazione
2. Mantenimento del carrello negli acquisti online
3. Creazione di raccomandazioni (come ad esempio le pubblicità mirate)

5.8.1 L'architettura dei Cookie

L'intero meccanismo di funzionamento dei cookie si basa su 4 componenti fondamentali che lavorano in sinergia:

1. Una **riga di intestazione specifica presente nella risposta HTTP** generata dal server
2. Una **riga di intestazione specifica inserita nella successiva richiesta HTTP** da parte del client.
3. Un **file locale relativo al cookie**, conservato sul dispositivo del client e gestito dal suo browser.
4. Un **database di backend** mantenuto e gestito attivamente dal Web server

5.9 Web Caching

L'obiettivo principale del Web caching è soddisfare le richieste dei client utilizzando delle copie salvate localmente, invece di dover scaricare i file dal server Web di origine.

Queste copie possono risiedere in 2 posti:

1. Nella **Web cache locale del dispositivo**, gestita direttamente dal browser dell'utente
2. In appositi server disseminati nelle reti istituzionali o di accesso, noti come **Web cache** o **server proxy**

5.9.1 Il ruolo del Server Proxy

Quando un proxy è attivato, il browser invia tutte le sue richieste HTTP direttamente a questo nodo intermedio.

Il server proxy svolge un doppio ruolo:

- **Si comporta da Server**

Se possiede già in memoria l'oggetto richiesto dal client, lo invia immediatamente

- **Si comporta da Client**

Se l'oggetto non è presente nella sua cache, lo richiede al server Web di origine, lo salva e infine lo inoltra al client che lo aveva richiesto

È il server di origine a dettare le regole alla cache: attraverso intestazioni specifiche nella risposta HTTP, comunica se un file può essere salvato e per quanto tempo

5.10 GET condizionale

Il problema principale delle cache è capire se il file salvato in memoria è ancora la versione più recente, oppure se il server di origine lo ha aggiornato.

Per risolvere questo problema, il protocollo HTTP utilizza una richiesta speciale detta "GET condizionale".

5.10.1 L'intestazione "if-modified-since"

Quando il client (o il proxy) richiede un file che ha già in cache ma di cui vuole verificare la validità, inserisce nella richiesta HTTP un parametro specifico: `if-modified-since: <data>`

Con questa riga, il client sta dicendo al server:

"Inviarmi questo file solo se è stato modificato dopo questa precisa data (ovvero la data in cui ho scaricato la mia copia locale)"

Possibili risposte del Server

A fronte di una GET condizionale, il server Web controlla il file originale e può rispondere in due modi:

- **Se il file NON è stato modificato**

Il server genera un messaggio di risposta con il codice di stato 304 Not Modified.

La particolarità fondamentale è che **questa risposta è vuota**, ovvero non include alcun oggetto al suo interno.

Questo permette di risparmiare un'enorme quantità di banda e tempo di trasferimento, autorizzando il client a usare la copia che ha già in memoria.

- **Se il file È stato modificato**

Il server elabora la richiesta come una normale GET.

Risponde con un codice 200 OK e inserisce nel corpo del messaggio la nuova versione aggiornata dei dati.

5.11 Message vs Streaming communication

La differenza principale tra la comunicazione a messaggi e quella a stream risiede nel modo in cui i dati vengono elaborati e percepiti dai diversi livelli dell'architettura di rete

5.11.1 Livello di Applicazione (Comunicazione a Messaggi)

A livello applicativo, come nel caso del protocollo HTTP, la comunicazione è **orientata al messaggio**.

Questo significa che i software comunicano scambiandosi pacchetti di dati interi e ben definiti (i messaggi).

5.11.2 Livello di Trasporto (Comunicazione a Stream e Segmenti)

Per il livello di trasporto, i messaggi non esistono come entità logiche, ma sono visti nel loro complesso come un **flusso di byte da trasportare in rete**.

Il livello di trasporto prende questo stream e lo scompone in **segmenti**.

La gestione di questi segmenti dipende dal protocollo utilizzato:

- **Servizio UDP**

Il protocollo scompone lo stream di byte in segmenti e li inoltra al livello di rete in **modalità best-effort**

- **Servizio TCP**

Anche il TCP scompone e invia lo stream come segmenti, ma **ogni segmento viene numerato**.

Questa numerazione permette a TCP di garantire 3 funzionalità cruciali:

1. **Riordinamento**

I segmenti arrivati fuori sequenza vengono rimessi in ordine giusto.

2. **Controllo delle duplicazioni**

I segmenti ricevuti più volte vengono scartati

3. **Controllo delle perdite**

Il sistema si accorge dei buchi nella numerazione e richiede il rinvio dei segmenti mancanti

5.12 Tipi di comunicazione Message-oriented communication

La differenza fondamentale tra i vari tipi di comunicazione si basa sul **punto di sincronizzazione**, ovvero il momento esatto in cui il client smette di attendere una conferma e riprende regolarmente la propria esecuzione.

5.12.1 Comunicazione Persistente Asincrona

- **Punto di sincronizzazione**

Avviene all'**invio della richiesta**

- **Funzionamento**

Il mittente sospende la sua esecuzione solo per il tempo necessario affinché il messaggio venga preso in carico dal proprio sistema intermedio locale (il middleware lato mittente), e non attende alcuna risposta dal destinatario finale

- **Esempio**

Il classico sistema di posta elettronica, in cui il client attende solo che il proprio server SMTP accetti l'email da spedire.

5.12.2 Comunicazione persistente sincrona

- **Punto di sincronizzazione**

Avviene all'inoltro della richiesta

- **Funzionamento**

Il mittente invia il messaggio e attende.

Riprenderà la sua esecuzione solo quando il software middleware situato fisicamente sul lato del ricevente avrà ricevuto il messaggio e avrà inviato una notifica di avvenuta consegna.

5.12.3 Comunicazione transiente sincrona basata sulla consegna (Delivery based)

- **Punto di sincronizzazione**

Avviene all'inizio dell'**elaborazione della richiesta**(at request processing)

- **Funzionamento**

Il mittente rimane bloccato in attesa finché l'applicazione destinataria (che deve essere attiva) gli notifica di aver effettivamente iniziato a elaborare la richiesta ricevuta.

- **Esempio**

L'invio di comandi a un server remoto tramite protocollo SSH

5.12.4 Comunicazione transiente sincrona basata sulla risposta (Response based)

- **Punto di sincronizzazione**

Avviene dopo l'elaborazione della richiesta (after processing)

- **Funzionamento**

È il modello che richiede l'attesa maggiore.

Il mittente rimane in sospeso e continua l'esecuzione solo quando il destinatario ha elaborato con successo e in modo completo la richiesta, restituendo la risposta finale.

- **Esempio**

Il protocollo HTTP in cui il browser attende di ricevere dal server la risorsa richiesta.

5.13 Modello a Code di Messaggi (Message Queuing)

In questo modello, le applicazioni non comunicano passandosi i dati direttamente, ma inseriscono i messaggi all'interno di code specifiche gestite da appositi gestori (queue manager)

Questa architettura si basa su 2 caratteristiche fondamentali:

- **Persistenza (Storage)**

Il sistema ha la capacità di memorizzare fisicamente i messaggi.

Una volta che un messaggio viene inserito all'interno di una coda, vi rimane al sicuro finché non viene esplicitamente prelevato e rimosso.

- **Disaccoppiamento temporale (Asincronismo)**

Non è assolutamente necessario che il mittente e il destinatario siano in esecuzione nello stesso momento.

Il mittente può generare un messaggio che se il ricevente è spento e, viceversa, il ricevente può recuperarlo dalla sua coda molto tempo dopo che il mittente ha finito di lavorare

5.13.1 Operazioni per interagire con la coda

Per interagire con queste code, l'interfaccia di comunicazione fornisce **4 primitive di base**.

1. **PUT**

Inserisce un messaggio nella coda di invio (operazione non bloccante).

2. **GET**

Rimuove il primo messaggio disponibile dalla coda di ricezione (operazione bloccante, il ricevente si ferma ad aspettare finché non arriva un messaggio).

3. **POLL**

Controlla in modo non bloccante la presenza di messaggi e, se ci sono, rimuove il primo.

4. **NOTIFY**

Installa un "handler" (una funzione di callback) che viene richiamato automaticamente dal middleware non appena un nuovo messaggio inserito nella coda avvisando l'applicazione.

5.14 Message Broker e Pattern Publish/Subscribe

Il pattern Publish/Subscribe è un'evoluzione che utilizza un nodo centrale chiamato **Message Broker** per gestire lo smistamento delle informazioni, basandosi sugli **argomenti** (Topic) anziché sugli indirizzi di destinazione.

L'architettura funziona in questo modo:

- **I Publisher (pubblicatori)**

Non sanno chi riceverà i loro dati, si limitano a collegarsi al broker dichiarando:

"voglio inviare messaggi relativi a questo specifico Topic".

- **I Subscriber (sottoscrittori)**

A loro volta, si registrano al broker dichiarando:

"voglio ricevere tutti i messaggi pubblicati su questo Topic".

Questa meccanica garantisce una **totale indipendenza** tra chi produce i dati e chi li consuma, rendendo facilissimo creare reti di comunicazione complesse del tipo uno-a-molti, molti-a-uno o molti-a-molti.

Se per un certo topic non ci sono subscriber attivi in quel momento, il broker può mantenere i dati in memoria applicando diverse politiche.

5.14.1 Funzionalità speciali per i sistemi critici (es. protocollo MQTT)

Poiché il modello pub/sub è massicciamente utilizzato nell'Internet of things e in ambienti critici, i broker offrono funzionalità avanzate:

- **Messaggio "Retain"**

Il publisher può chiedere al broker di conservare uno e un solo messaggio per un topic.

Grazie al flag retain, non appena un nuovo subscriber si connette, riceve immediatamente quest'ultimo valore valido senza dover aspettare una nuova pubblicazione.

- **Last Will and Testament (LWT)**

Al momento della connessione, il publisher può consegnare al broker un "testamento".

Se il dispositivo publisher dovesse subire una disconnessione inaspettata, il broker inoltrerà automaticamente questo messaggio di allarme a tutti i subscriber.

È una funzione fondamentale per accorgersi tempestivamente di un'anomalia nei sistemi critici.

Capitolo 6

Formati di scambio

6.1 Perché usarli?

Nei sistemi distribuiti, le diverse componenti sono spesso sviluppate in linguaggi di programmazione differenti e vengono eseguite su sistemi operativi o piattaforme eterogenee.

Poiché questi sistemi non dividono la stessa memoria fisica, i dati devono necessariamente viaggiare sulla rete sotto forma di un flusso ininterrotto di byte.



Figura 6.1: Procedura di comunicazione tra client e server attraverso formati di scambio

6.2 Come usarli?

Per far sì che sistemi che non si conoscono possano comunicare e capirsi, è fondamentale concordare un "contratto" comune, ovvero un formato di scambio dei dati che sia totalmente indipendente dalla piattaforma utilizzata, come ad esempio i formati di testo XML o JSON.

Per trasferire attraverso la rete delle strutture dati complesse in modo sicuro, le applicazioni si avvalgono di due operazioni speculari:

6.2.1 Marshaling (o Serializzazione)

È il processo mediante il quale l'applicazione **mittente** converte le proprie strutture dati residenti in memoria in un formato trasmissibile (come testo o byte) per poi inviarlo sulla rete o salvarlo su un file.

6.2.2 Unmarshaling (o Deserializzazione)

È il processo inverso eseguito dal **destinatario**. Ricevuto il flusso di dati serializzato, il sistema ricevente lo "decompacchetta" e ricostruisce in memoria la struttura dati logica originale per poterla elaborare correttamente.

Capitolo 7

Linguaggi di Markup

7.1 Definizione

Sono sistemi di annotazione che usano speciali etichette (tag) per descrivere in modo esplicito la semantica e la struttura dei documenti.

7.2 Tipi di marcatura

Esistono 3 tipologie principali di marcatura:

- **Descrittive**

Etichettano semplicemente parti del testo, disaccoppiando la struttura dalla presentazione del testo stesso.

- **Procedurali**

Definiscono istruzioni per programmi che elaborino il testo al quale sono associate.

- **Presentazione**

Definiscono come visualizzare il testo al quale sono associate.

Capitolo 8

HTML - HyperText Markup Language

8.1 Definizione

L'HTML è un linguaggio di marcatura basato su etichette che servono ad annotare un documento in modo che queste annotazioni siano sintatticamente distinguibili dal testo vero e proprio.

8.2 Cosa fa?

Lo scopo principale dell'HTML è quello di fornire la struttura logica e descrivere la semantica dei contenuti presenti in una pagina Web.

Il suo ruolo all'interno dell'ecosistema Web è strettamente legato al concetto di **separazione delle responsabilità**, l'HTML ha il compito esclusivo di strutturare gerarchicamente l'informazione, garantendo la totale separazione tra il contenuto e la sua presentazione, la quale viene delegata a CSS.

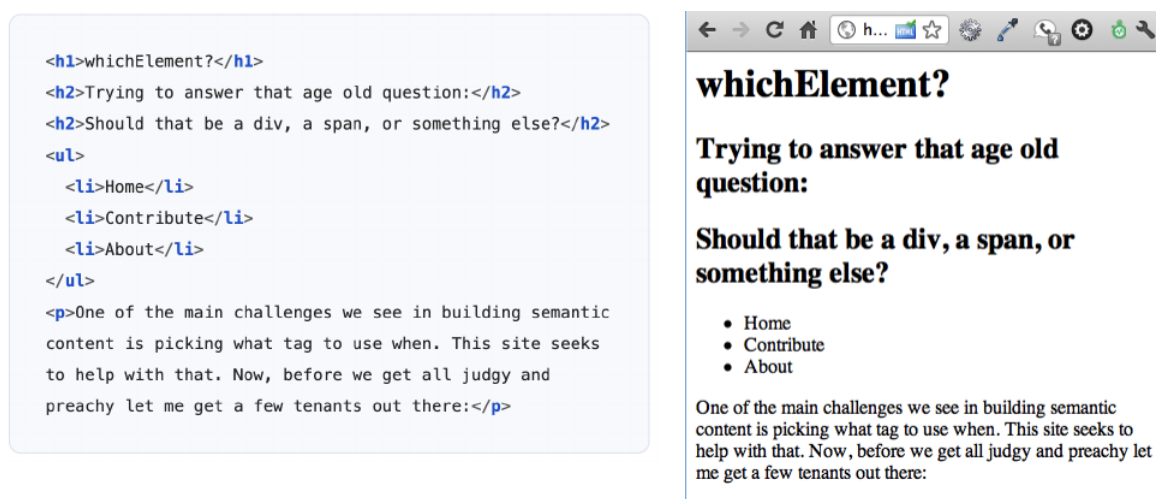


Figura 8.1: Esempio di codice HTML e il suo risultato

8.3 Uso dei Tag e la semantica

L'HTML struttura il documento racchiudendo le parti di testo all'interno di specifiche etichette di apertura e chiusura.

L'uso corretto di questi tag conferisce un significato preciso al testo contenuto

Tag	Significato
<h1>--<h6>	Intestazioni gerarchiche
<p>	Paragrafo
, ,	Liste (non ordinata, ordinata, elemento)
<table>	Dati tabulari
<form>, <input>	Form interattivi
<a>	Collegamento ipertestuale
, 	Enfasi semantica
 e sono semantici (significano <u>enfasi</u> e importanza), a differenza di <i> e che sono solo visivi (corsivo e grassetto senza significato).	

8.4 Interazione con l'utente: Forum

Quando è necessario richiedere dati all'utente, l'HTML utilizza una struttura specifica chiamata forma, definita dal tag `<form>`.

All'interno del form, l'HTML mette a disposizione svariati widget di interfaccia, come caselle di testo, checkbox, bottoni e menu a tendina

Una volta raccolti i dati, il form li invia a uno script lato server (il cui indirizzo è specificato nell'attributo `action`). L'invio (definito dall'attributo `method`) avviene principalmente in 2 modalità:

- **Metodo GET**

Accoda i parametri inseriti dall'utente direttamente alla fine dell'URL della pagina.

- **Metodo POST**

Inserisce i dati in modo invisibile all'interno del corpo della richiesta HTTP.

Esempio codice di un form

```
1 <!DOCTYPE html>
2 <html>
3 <body>
4
5 <h2>HTML Forms</h2>
6
7 <form action="/action_page.php">
8   <label for="fname">First name:</
   label><br>
9   <input type="text" id="fname"
   name="fname" value="John"><br>
10  <label for="lname">Last name:</
   label><br>
11  <input type="text" id="lname"
   name="lname" value="Doe"><br><br>
12  <input type="submit" value="
   Submit">
13 </form>
14
15 <p>If you click the "Submit" button
   ...</p>
16
17 </body>
18 </html>
```

Example

First name:

Last name:

[Try it Yourself »](#)

8.5 DOM - Document Object Model

DOM è un'interfaccia neutrale rispetto al linguaggio di programmazione e alla piattaforma utilizzata, per consentire ai programmi l'accesso e la modifica dinamica di contenuto, struttura e stile di un documento Web.

8.5.1 Struttura ad Albero

Quando il browser riceve la pagina HTML, non la lascia sotto forma di testo, ma la converte in una complessa struttura ad albero di oggetti.

All'interno di questo albero, ogni singolo tag HTML si trasforma in un "nodo", stabilendo così delle **relazioni gerarchiche di parentela**.

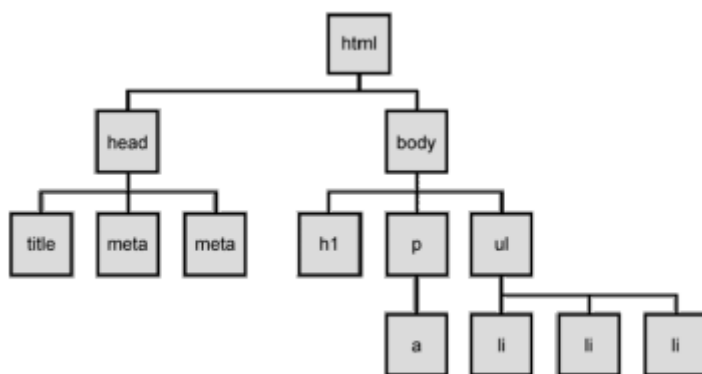


Figura 8.2: Esempio di albero DOM

8.5.2 Navigazione dei nodi

Per permettere ai linguaggi di scripting di muoversi agilmente all'interno di questa architettura, il DOM mette a disposizione diverse proprietà specifiche:

- **parentNode**: Raggiunge il nodo genitore diretto.
- **childNodes**: Restituisce la lista di tutti i nodi figli.
- **firstChild** e **lastChild**: Puntano rispettivamente al primo e all'ultimo figlio del nodo.
- **nextSibling** e **previousSibling**: Permettono di spostarsi orizzontalmente sul nodo fratello successivo o su quello precedente.

8.5.3 Identificazione dei Nodi

Prima di poter manipolare un elemento della pagina, è necessario "trovarlo" all'interno dell'albero.

Ogni nodo può essere caratterizzato da attributi utili a questo scopo:

- Un **identificatore univoco (ID)**, anche se il DOM non ne garantisce l'assoluta unicità a livello tecnico
- Una **classe**, che indica l'appartenenza a un gruppo specifico

8.5.4 Funzioni di ricerca per i nodi

Per sfruttare questi attributi, il browser stesso fornisce nativamente delle funzioni di ricerca rapida, tra cui le più importanti sono:

- **getElementById(idName)**: per estrarre il singolo nodo associato a quel preciso ID.
- **getElementsByClassName(className)**: per estrarre tutti i nodi associati a una specifica classe.

8.5.5 Eventi e Listener

Il DOM non serve solo a modificare passivamente la pagina, ma anche a farla reagire alle azioni dell'utente.

Questo avviene tramite gli **eventi**, ovvero dei segnali emessi dal sistema quando accade qualcosa nel documento.

Javascript può mettersi in ascolto di questi eventi e reagire di conseguenza agganciato delle funzioni specifiche chiamate **listener**.

Categoria	Eventi principali
Mouse	click, dblclick, mouseover, mouseout
Tastiera	keydown, keyup, keypress
Form	submit, change, focus, blur, input
Documento	DOMContentLoaded, load, resize, scroll
Touch	touchstart, touchend, touchmove

```
1 // Seleziono un elemento
2 const btn = document.getElementById("myBtn");
3
4 // Aggiungo un listener
5 btn.addEventListener("click", function(e){
6     console.log("cliccato", e.target);
7 })
```

Esempio di codice

Capitolo 9

CSS - Cascade Style Sheet

9.1 Definizione

Il CSS è il linguaggio utilizzato per definire lo stile e la presentazione visiva dei contenuti web.

Il suo obiettivo primario è concretizzare il principio di "separation of concerns" garantendo una netta divisione tra il **contenuto** e la **vesta estetica**

9.2 Modalità di inclusione

Il codice CSS può essere applicato a un documento HTML i 3 modi differenti:

- **Inline**

Inserito direttamente sul singolo tag HTML tramite l'attributo `style=""`.

- **Interno**

Definito all'interno di un blocco di tag `<style>` situato nella sezione `<head>` della pagina.

- **Esterno**

Scritto all'interno di un file `.css` separato e collegato alla pagina HTML tramite il tag `<link rel="stylesheet" href="theme.css">`.

Questo è il modello preferito perchè separa fisicamente contenuto e stile.

9.3 Struttura Base

Una regola CSS è composta da un "selettore" seguito da un blocco di dichiarazione racchiuse tra parentesi graffe.

Ogni dichiarazione è una coppia proprietà/valore (selettoreproprietà: valore;)

Esempio codice CSS

```
1 h1{
2   color: red;
3   font-size: 2em;
4 }
```

9.3.1 Selettori

I selettori sono il meccanismo con cui il CSS individua precisi elementi o gruppi di elementi nell'albero DOM a cui applicare il design.

Selettori fondamentali

- **Elemento (Tag)**: Seleziona tutti gli elementi di un certo tipo.
- **Classe**: Individua gli elementi che possiedono uno specifico attributo `class`, preceduto da un punto.
- **ID**: Punta a un elemento univoco contrassegnato da uno specifico attributo `id`, preceduto dal cancelletto.
- **Universale**: L'asterisco (`* . . .`) permette di selezionare indiscriminatamente tutti gli elementi della pagina

9.3.2 Combinatori

I combinatori servono per selezionare elementi non per il tag che sono, ma per la loro posizione all'interno dell'albero DOM

- **Discendente**: `div p` seleziona tutti i paragrafi contenuti dentro un `div`.
- **Figlio diretto**: `ul > li` seleziona esclusivamente gli elementi della lista che sono figli diretti del tag `ul`
- **Gruppo**: `h1, h2, h3` consente di raggruppare selettori diversi applicando la stessa regola in un colpo solo

9.3.3 Pseudo-classi

Un elemento HTML è sempre lo stesso tag, ma a livello di interfaccia può cambiare stato.

Le psuedo classi permettono di stilizzare questi stati dinamicamente, senza alterare l'HTML, esempi notevoli sono:

- **a: hover**
- **input: focus**
- **li: first-child**

9.4 Algoritmo "a Cascata"

Poichè esistono diversi fogli di stile, è del tutto normale che le regole si sovrappongano e generino dei conflitti.

Il termine "Cascading" si riferisce all'algoritmo che decide quale regola abbia la precedenza.

La priorit viene valutata secondo quest'ordine:

1. **Importanza**

L'eventuale utilizzo del flag esplicito !important associato a un attributo.

2. **Specificità**

Dipende dal settore utilizzato.

Un ID è più specifico di una classe, che a sua volta prevale sul selettore di un semplice elemento.

3. **Ordine nel sorgente**

A parità di specificità, vince la regola dichiarata per ultima nel file.

9.5 Media Query

Per far si' che la grafica si adatti ai diversi dispositivi, si utilizzano le "media query".

Questi costrutti CSS applicano stili in modo condizionale verificando le caratteristiche del dispositivo in uso, come la larghezza, l'altezza, la risoluzione e l'orientamento dello schermi.

Ad esempio, @media screen and (min-width: 480px)

Capitolo 10

XML - eXtensible Markup Language

L'XML si è imposto come standard de facto per lo scambio di informazioni semi-strutturate.

Inizialmente pensato come potenziale successore dell'HTML per il Web, è diventato un linguaggio generico e universale.

10.1 Caratteristiche dei Documenti XML

- **Marcatura descrittiva** e non procedurale (descrive la struttura logica).
- Marcatura definisce la **struttura logica** del documento.
- **Flessibilità**: le etichette possono cambiare in base all'applicazione.
- Il **tipo di documento** specifica la struttura della marcatura ammessa.

10.2 Documenti "Well-formed" (Ben Formati)

Un documento testuale che rispetta rigorosamente le regole sintattiche di base della specifica XML è definito **well-formed**.

Affinchè un documento ottenga questo status, deve soddisfare determinati requisiti:

- **Presenza di elementi**: Il documento deve sempre contenere almeno un elemento.
- **Singolo Elemento Radice (Root)**: Deve esistere esattamente un solo elemento radice che fa da "contenitore" generale, includendo al suo interno tutti gli altri elementi del documento.
Nessuna parte di questo elemento radice può apparire come contenuto di un altro elemento.
- **Etichette di apertura e chiusura**: Ogni elemento deve essere perfettamente delimitato.
È costituito da una coppia di etichette e da tutto il contenuto che queste racchiudono.
- **Annidamento corretto**: Gli elementi che costituiscono il documento devono essere correttamente annidati l'uno dentro l'altro.

Questo significa che i tag non possono accavallarsi: un elemento figlio deve essere chiuso prima che venga chiuso il suo elemento padre.

- **Validità delle entità**: Ogni singola entità referenziata nel documento, sia direttamente che indirettamente, deve essere a sua volta well formed.

10.2.1 Esempio di documento XML

Esempio di documento XML Well-formed

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <libreria>
3   <libro categoria="romanzo">
4     <titolo>Il Signore degli Anelli</titolo>
5     <autore>J.R.R. Tolkien</autore>
6     <anno>1954</anno>
7     <prezzo valuta="EUR">25.50</prezzo>
8   </libro>
9
10  <libro categoria="informatica">
11    <titolo>Imparare XML</titolo>
12    <autore>Mario Rossi</autore>
13    <anno>2023</anno>
14    <prezzo valuta="EUR">15.00</prezzo>
15  </libro>
16 </libreria>
```

10.3 DTD (Document Type Definition)

Un documento well-formed possiede una struttura arbitraria: i nomi dei tag e gli attributi sono liberi. Per definire quali strutture sono ammesse per una specifica classe di documenti, si utilizza una grammatica chiamata DTD.

A livello di posizionamento, una DTD può essere interna al documento XML oppure esterna

10.3.1 Struttura DTD

La struttura di una DTD si basa principalmente su 3 tipi di dichiarazione che stabiliscono le regole del documento

- **Tipi di Elemento**

Definisce quali tag possono esistere come possono essere annidati gli uni dentro agli altri.

Per definire le regole di annidamento e molteplicità, la DTD utilizza dei simboli che funzionano in modo del tutto simile alle espressioni regolari:

- , (virgola): Indica una sequenza ordinata
- | (pipe): Indica un'alternativa
- ?: L'elemento è opzionale
- +: L'elemento deve apparire una o più volte.
- *: L'elemento può apparire 0 o infinite volte.
- Se un elemento non contiene altri tag ma solo del testo libero con la parola chiave speciale #PCDATA

- **Attributi <!ATTLIST ...>**

Dichiara quali attributi sono consentiti per un determinato elemento, di che tipo sono

- **Entità e Notazioni**

Definiscono abbreviazioni utilizzabili nel documento e a formati esterni

10.3.2 Esempio di DTD

Esempio di DTD

```
1 <!ELEMENT media (cds | books)>
2 <!ELEMENT cds (cd*)>
3 <!ELEMENT cd (title, artist, album+)>
4 <!ATTLIST cd quantity CDATA #REQUIRED>
5 <!ELEMENT title (#PCDATA)>
6 <!ELEMENT artist (#PCDATA)>
7 <!ELEMENT album (#PCDATA)>
```

10.3.3 I Limiti delle DTD

Sebbene le DTD vengano utilizzate per definire la "grammatica" di un file XML, esse descrivono solo regole strutturali di base e non permettono di definire la struttura dettagliata o i tipi di dato.

Queste impostazioni porta ad alcune evidenti limitazioni:

- **Assenza di vincoli sui tipi di dato**
- **Mancanza di vincoli di co-occorrenza**

Non si possono creare regole condizionali legate alla presenza di altri elementi. Ad esempio, non è possibile specificare che l'elemento "unit" debba essere ammesso solo quando è presente anche l'elemento "amount".

- **Flessibilità dello schema limitata**
- **Assenza di architettura avanzate**

Le DTD non supportano l'ereditarietà (subtyping), impedendo di fatto il riuso delle definizioni.

- **Mancanza di chiavi composite**

10.4 XML Schema

10.4.1 Definizione

Per affrontare e superare le pesanti limitazioni delle DTD, è stato introdotto l'**XML Schema**, un linguaggio di schema decisamente più sofisticato.

Le sue caratteristiche principali includono:

- **Tipizzazione forte**

A differenza delle DTD, supporta nativamente la tipizzazione dei valori.

Permette inoltre di applicare vincoli restrittivi, potendo definire un minimo e un massimo per i valori inseriti.

- **Strutture complesse e chiavi esterne**

Consente agli sviluppatori di definire tipi di dato completi e supporta molte funzionalità avanzate, tra cui l'uso di vincoli di unicità, chiavi e il supporto all'ereditarietà.

- **Standardizzazione e sintassi**

Mentre le DTD utilizzano una propria grammatica a simboli, XML Schema è scritto utilizzando la normale sintassi XML.

Questo gli garantisce una rappresentazione più standard e una perfetta integrazione con i namespace.

- **Elevata complessità**

Il compromesso per avere uno strumento così potente è che l'XML Schema risulta molto più verboso e significativamente più complesso rispetto a una DTD. Per definire la struttura, richiede infatti l'uso di numerosi tag annidati e costrutti formali.

Capitolo 11

JSON - Javascript Object Notation

11.1 Definizione

Il JSON è un formato testuale leggero impiegato per lo scambio di dati.

Le sue caratteristiche lo rendono universalmente diffuso perchè è facile da leggere e scrivere per gli esseri umani, e al contempo facile da analizzare e generare per i computer.

11.2 Strutture dati universali

Praticamente tutti i moderni linguaggi di programmazione supportano il formato JSON perchè esso si fonda su sole due strutture dati universali:

11.2.1 Oggetti

Sono insiemi non ordinati di coppie nome/valore.

- Si utilizzano tipicamente quando i nomi delle chiavi sono stringhe arbitrarie
- Sintatticamente, un oggetto inizia e finisce con parentesi graffe ({ })
- Ogni nome di chiave è seguita dai punti (:) e separato dal suo valore, mentre le varie coppie sono separate da virgole (,)

Esempio di oggetto JSON

```
1 {  
2   "nome": "Mario",  
3   "eta": 30,  
4   "attivo": true,  
5   "eliminato": false,  
6   "note": null,  
7   "indirizzo": {  
8     "citta": "Milano"  
9   },  
10  "tag": ["web", "json"]  
11 }
```

11.2.2 Array

Sono collezioni ordinate di valori

- Vengono preferiti quando i nomi delle chiavi sono considerati interi sequenziali.
- Iniziano e finiscono con le parentesi quadre ([]) e i valori al loro interno sono separati da virgole

Esempio di oggetto contenente array

```
1 [
2   {
3     "nome": "Mario Rossi",
4     "eta": 30
5   },
6   {
7     "nome": "Anna Bianchi",
8     "eta": 25
9   }
10 ]
```

11.3 Tipi di valore supportati

Il "valore" associato a una chiave all'interno di un JSON può assumere solo forme molto specifiche:

- **String**: Testo UNICODE racchiuso rigorosamente tra doppie virgolette ("string").
- **Number**: sia interi che reali a virgola mobile.
- **Boolean**: true o false.
- **null**: valore nullo.
- Un altro **Oggetto** o un altro **Array** permettendo così di creare strutture complesse e fortemente annidate.

11.4 JSON Schema

11.4.1 Definizione

Il JSON Schema è un vero e proprio vocabolario, scritto anch'esso in formato JSON, il cui scopo è descrivere e validare la struttura di altri documenti JSON.

Nel contesto dei sistemi distribuiti e dello sviluppo software, viene impiegato principalmente per 3 scopi principali:

- **Validare i dati in ingresso**
Controllare che le informazioni in arrivo rispettino i formati attesi prima di farli processare dalla logica applicativa.
- **Documentare**
Formalizzare e chiarire i "contratti" di comunicazione e di scambio dati tra sistemi differenti.

- **Generare codice**

Aiutare a generare automaticamente interfacce o porzioni di codice basandosi sulla precisa struttura descritta.

Esempio di JSON Schema

11.4.2 Esempio

```
1 {  
2   "$schema": "http://json-schema.org/draft-07/schema",  
3   "type": "object",  
4   "properties": {  
5     "nome": { "type": "string" },  
6     "eta": { "type": "number", "minimum": 0 },  
7     "attivo": { "type": "boolean" }  
8   },  
9   "required": ["nome", "eta"]  
10 }
```

Capitolo 12

Applicazioni Web

12.1 Web statico

Il web delle origini era un sistema completamente statico: il server riceveva la richiesta e si limitava a restituire documenti HTML salvati su disco "così com'erano", senza operare alcuna elaborazione dinamica per gli utenti.

Il primo passo per la comunicazione attiva dal client verso il server si è avuto con l'introduzione dei **form HTML**.

12.2 Comunciazione attraverso HTTP

La comunicazione avviene tramite il protocollo HTTP, che presenta queste caratteristiche chiave:

- È un formato basato su testo composto da Header e Body.
- Consente di generalizzare la natura dei file in transito utilizzando un identificativo del formato chiamato payload MIME
- È "stateless", ovvero non conserva alcuna memoria tra le chiamate: ogni richiesta HTTP è un messaggio autonomo completamente slegato dai precedenti

Capitolo 13

Sviluppo lato Client - HTML

13.1 Interazione con il server basate su link

Il metodo più basilare per l'interazione è l'utilizzo dei link (<a>).

Un link punta a una specifica risorsa remota sul server e, quando viene cliccato, fa in modo che il browser generi ed invii in automatico una richiesta HTTP di tipo GET all'URL specificato.

Esempio di link che punta ad una risorsa

```
1 <p>Click the link to ask the servlet to send back an HTML document</p>
2 <a href="http://localhost:8080/SlideServlet/GetHTTPServlet">
3   Get HTML Document
4 </a>
```

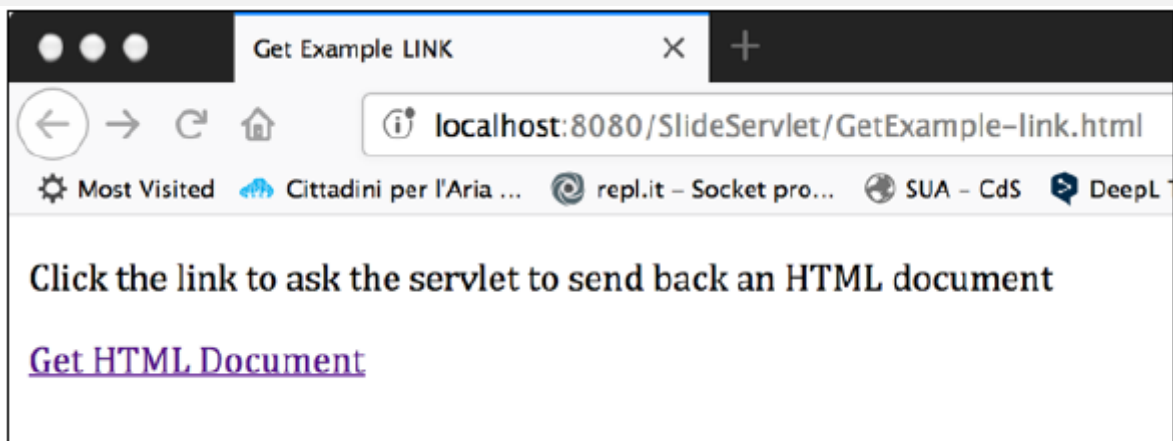


Figura 13.1: Risultato grafico

13.2 Interazioni basate su form

I Form (<text>) sono lo strumento HTML dedicato a richiedere e raccogliere input dall'utente per poi inviarlo al server.

A livello di comunicazione HTTP, un form può operare principalmente in due modi:

13.2.1 Form con GET

Se il form utilizza il metodo GET, i valori inseriti dall'utente vengono agganciati direttamente all'URL della richiesta sotto forma di Query parameter

Esempio di form con metodo GET

```
1 <form action="http://localhost:8080/SlideServlet/GetHTTPServlet" method="GET">
2   <p>Click the button to have the servlet send an HTML document</p>
3   <input type="submit" value="Get HTML Document">
4 </form>
```

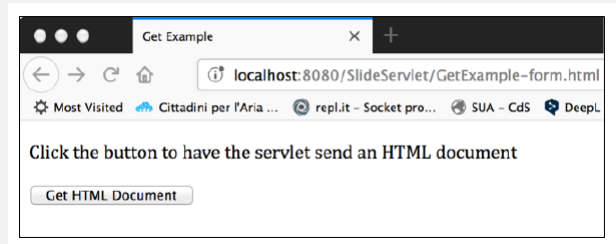


Figura 13.2: Risultato grafico

13.2.2 Form con POST

Se il form utilizza il metodo POST, i dati vengono invece incapsulati in modo più nascosto all'interno del payload (il corpo centrale del messaggi HTTP), mantenendo l'URL pulito.

Esempio di form con metodo POST

```
1 <form action="http://localhost:8080/SlideServlet/PostHTTPServlet" method="POST">
2   What is your favorite pet?<br> <br>
3   <input type="radio" name="animal" value="dog">Dog<br>
4   <input type="radio" name="animal" value="cat">Cat<br>
5   <input type="radio" name="animal" value="bird">Bird<br>
6   <input type="radio" name="animal" value="snake">Snake<br>
7   <input type="radio" name="animal" value="none" checked>None<br>
8   <br> <input type="submit" value="Submit"> <input type="reset">
9 </form>
```

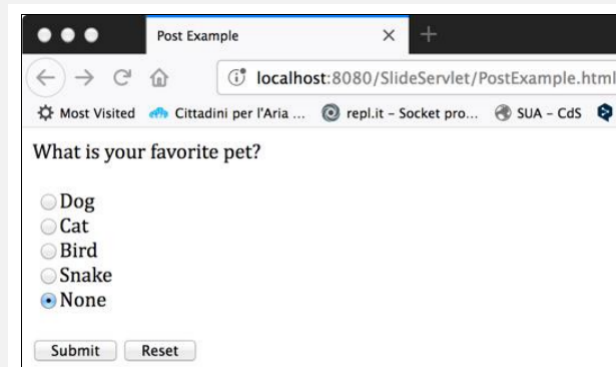


Figura 13.3: Risultato grafico

13.2.3 Limiti dei Form

I form HTML nativi possiedono un limite importante: supportano solamente 3 tipi di codifica standard (i cosiddetti enctype) e non contemplano l'uso nativo di formati strutturati come JSON o XML

Capitolo 14

Sviluppo lato Server - Servlet Java

14.1 Definizione

Le servlet sono piccole applicazioni e oggetti scritti in Java che vivono all'interno della memoria di un server.

Non funzionano in modo autonomo, ma vengono gestite da un ambiente di esecuzione detto Servlet Container (o application server, come Apache Tomcat), il quale si fa carico di accenderle, segnerle e inoltrare loro il traffico di rete.

14.2 Configurazione di una servlet

Le servlet richiedono una configurazione che definisca:

- Nome della servlet
- Classe di implementazione
- Mapping URL
- Parametri di inizializzazione

14.3 Metodi

14.3.1 Metodi del ciclo di vita

A livello di codice, ogni Servlet implementa l'interfaccia `jakarta.servlet.Servlet` che possiede come metodi:

- `void init(ServletConfig config)`

Inizializza la servlet, viene invocata dopo la creazione della stessa.

Esempio di metodo `init`

```
1 @WebServlet(  
2     urlPatterns = "/LaMiaServlet",  
3     initParams = {  
4         @WebInitParam(name = "emailAmministratore", value = "admin@miosito.
```

```

        it")
    }
}
)
public class MiaServlet extends HttpServlet {

    private String emailDiContatto;

    @Override
    public void init(ServletConfig config) throws ServletException {
        super.init(config);

        // E' buona norma richiamare il metodo della superclasse
        emailDiContatto = config.getInitParameter("emailAmministratore");

        System.out.println("Servlet caricata. Email impostata a: " +
            emailDiContatto);
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse
        response) {
        // Qui la Servlet puo' usare la variabile emailDiContatto
    }
}

```

- void destroy()

Chiamata quando la servlet termina (es: per chiudere un file o una connessione con un data-base)

Esempio del metodo destroy

```

1 public class ShutdownServlet extends HttpServlet {
2
3     // 1. Inizializzazione
4     @Override
5     public void init() throws ServletException {
6         System.out.println("Servlet avviata e pronta.");
7     }
8
9     // 2. Servizio (Eseguito ad ogni richiesta)
10    @Override
11    protected void doGet(HttpServletRequest req, HttpServletResponse res)
12        throws IOException {
13        res.getWriter().println("Servlet in esecuzione...");
14    }
15
16    // 3. Distruzione (Eseguito una sola volta alla chiusura)
17    @Override
18    public void destroy() {
19        // Qui si chiudono connessioni al DB o file aperti
20        System.out.println("Salvataggio dati in corso... Servlet rimossa.")

```

```

21     }
22 }

```

- `void service(ServletRequest request, ServletResponse response)`

Esempio del metodo service

```

1  public class MiaServletService extends HttpServlet {
2
3      /**
4       * Il metodo service() intercetta ogni richiesta.
5       * Nota: usa ServletRequest e ServletResponse (generici).
6       */
7      @Override
8      public void service(ServletRequest request, ServletResponse response)
9          throws ServletException, IOException {
10
11         response.setContentType("text/html");
12
13         // Questo messaggio apparira' sia per richieste GET che POST
14         response.getWriter().println("<html><body>");
15         response.getWriter().println("<h1>Richiesta intercettata dal metodo
16             service!</h1>");
17         response.getWriter().println("<p>Gestione universale senza
18             distinzione tra i verbi HTTP.</p>");
19         response.getWriter().println("</body></html>");
20     }
21 }

```

Invocato per gestire le richieste del client

- `ServletConfig getServletConfig()`
Restituisce i parametri di inizializzazione e il *ServletContext* che dà accesso all'ambiente.
- `String getServletInfo()`
Restituisce informazioni tipo autore e versione

14.3.2 Metodi classe `HttpServlet`

Nella pratica comune, si estende la classe `HttpServlet`.

Questa classe fornisce metodi specifici per gestire i diversi verbi del protocollo HTTP.

- `doGet(HttpServletRequest req, HttpServletResponse res)`
Gestisce le richieste di tipo GET.
È usato tipicamente per recuperare informazioni dal server senza modificarne lo stato

Esempio del metodo GET

```
1 public class SalutoServlet extends HttpServlet {
2
3     // Gestisce le richieste di lettura (GET)
4     @Override
5     protected void doGet(HttpServletRequest request, HttpServletResponse
        response)
6         throws ServletException, IOException {
7
8         // Prepariamo la risposta per il browser
9         response.setContentType("text/html");
10        PrintWriter out = response.getWriter();
11
12        // Inviemo il contenuto HTML
13        out.println("<html><body>");
14        out.println("<h1>Ciao!</h1>");
15        out.println("<p>Questa e' una risposta generata dal metodo doGet.</
            p>");
16        out.println("</body></html>");
17    }
18 }
```

- doPost(HttpServletRequest req, HttpServletResponse res)
Gestisce le richieste POST.
- doPut() / doDelete()

14.3.3 Metodi per gestire Richieste (HttpServletRequest)

Questi metodi permettono di accedere ai dati contenuti nell'header e nel body del messaggio HTTP inviato dal client:

- getParameter(String name)
Restituisce il valore associato al parametro specificato, tipicamente proveniente da una query string o da un form HTML
- getParameterNames()
Restituisce un'enumerazione contenente l'elenco di tutti i nomi dei parametri passati nella richiesta.
- getParametersValues(String name)
Restituisce un array di stringhe contenente tutti i valori associati a un determinato parametro.
- getCookies()
Restituisce un array contenente tutti i cookie memorizzati sul client e inviati al server con la richiesta.
- getSession(boolean create)
Restituisce l'oggetto HttpSession che identifica l'utente corrente per la gestione dello stato.

Se create è impostato a true, crea una nuova sessione qualora non ne esista già una.

14.3.4 Metodi per gestire le Risposte (HttpServletResponse)

Questi metodi permettono di configurare la risposta HTTP da inviare al client, impostando header, status code e il contenuto del body:

- `setContentType(String type)`

Specifica il tipo MIME del contenuto della risposta, in modo da comunicare al browser come deve interpretare e visualizzare i dati ricevuti.

- `getWriter()`

Restituisce uno stream di caratteri (oggetto `PrintWriter`) utilizzato per scrivere testo all'interno del corpo della risposta.

- `getOutputStream()`

Restituisce uno stream di byte (`ServletOutputStream`) utilizzato per scrivere dati binari nel corpo della risposta.

- `addCookie(Cookie cookie)`

Aggiunge un nuovo cookie all'interno dell'intestazione (header) della risposta, in modo che il browser del client lo memorizzi localmente.

14.4 Multithreading - Modello a Thread Pool e l'esecuzione

Il multithreading nella Servlet Java si basa su un'architettura fondamentale: molti thread attraversano una singola istanza della servlet.

A differenza di tecnologie storiche che creavano un processo separato per ogni utente, le servlet sono progettate per massimizzare la scalabilità.

Le servlet sono oggetti Java che risiedono nella memoria del server, ma non avviano thread per conto proprio: il codice dei loro metodi viene eseguito da thread creati e gestiti direttamente dal container.

Per ottimizzare i tempi di risposta, il container utilizza il pattern architetturale del **Thread pool**.

Questo significa che:

1. Viene pre-creato un gruppo fisso di thread che rimangono in attesa di lavoro.
2. Quando arriva una richiesta HTTP da un client, il container non istanzia una nuova servlet e non crea un nuovo thread da zero, ma preleva un thread dal pool e gli fa eseguire il metodo `doX` sulla singola istanza condivisa della servlet.
3. La dimensione del pool fissa un limite massimo al numero di thread che possono esistere contemporaneamente.

14.4.1 Vantaggi

Questa scelta progettuale ha diverse motivazioni che migliorano nettamente le prestazioni del server:

- **Risparmio di memoria:** Esistendo una sola istanza dell'oggetto servlet per tutti i client, si consuma una quantità di memoria minima.

- **Velocità:** Si abbattano i costi e i tempi legati all'inizializzazione ripetuta degli oggetti e alla creazione "on the fly" dei thread.
- **Persistenza:** Abilita la possibilità di mantenere in memoria risorse costose e condividerle tra varie richieste, come ad esempio un pool di connessioni a un database.

Capitolo 15

Sviluppo lato Server - Java Server Pages

15.1 Definizione

Le JSP (Java Server Pages) sono una tecnologia nata per facilitare la creazione di interfacce web dinamiche, separando la parte statica della pagina dalla logica applicativa.

La caratteristica più affascinante delle JSP è che non vengono interpretate direttamente.

Quando un client richiede una pagina JSP per la prima, il server la traduce automaticamente in codice Java, la compila in un file `.class`(bytecode) e la esegue come se fosse una normalissima Servlet.

15.2 Elementi di una pagina JSP

All'interno di un JSP si mescola in normale testo HTML con elementi specifici racchiusi in particolari tag.

Questi elementi si dividono in 3 categorie principali:

- **Direttive** (`<% @ %>`)

Danno istruzioni generali al server su come compilare la pagina.

Esempio JSP: Direttiva page

```
1 <%-- Direttiva page: imposta il tipo di contenuto e importa una classe --%>
2 <%@ page language="java" contentType="text/html; charset=UTF-8" import="
   java.util.Date" %>
3
4 <html>
5     <body>
6         <h1>Esempio Direttiva JSP</h1>
7         <p>
8             <%-- Usiamo la classe Date importata sopra --%>
9             La data e l'ora attuale sono: <%= new Date() %>
10        </p>
11    </body>
12 </html>
```

- **Azioni** (`<jsp:XXX>`)

Vengono processate a ogni singola richiesta e permettono di eseguire operazioni complesse senza scrivere codice Java puro:

Esempio di Azioni JSP

```
1 <html>
2 <body>
3   <h1>Pagina Principale</h1>
4
5   <!-- 1. Azione include: include il file al momento della richiesta --%>
6   <jsp:include page="data_corrente.jsp" />
7
8   <p>Il contenuto sopra e' stato incluso dinamicamente.</p>
9
10  <!-- 2. Esempio di forward (commentato per non bloccare la pagina)
11       Questa azione interrompe la pagina attuale e manda l'utente
12       altrove.
13   <jsp:forward page="login.jsp" />
14   --%>
15 </body>
16 </html>
```

• Elementi di Scripting

Permettono di inserire frammenti di codice Java vero e proprio.

Si dividono in:

- **Scriptlet** (<% %>) per istruzioni e cicli.
- **Declaration** (<%! %>) per dichiarare variabili o metodi.
- **Expression** (<%= %>) per valutare un'espressione e stamparne direttamente il risultato nell'HTML.

Esempio costrutti JSP

```
1 <%@ page language="java" contentType="text/html; charset=UTF-8" %>
2 <html>
3 <body>
4   <%!
5     // 1. DECLARATION (<%! %>): Dichiariamo una variabile e un metodo
6     private String titolo = "Lista Numerica";
7
8     public String trasformaInMaiuscolo(String testo) {
9       return testo.toUpperCase();
10    }
11   %>
12
13   <h1><%= trasformaInMaiuscolo(titolo) %></h1>
14   <!-- 2. EXPRESSION (<%= %>): Valuta e stampa il titolo in maiuscolo --%>
15   >
16   <ul>
```



```

17     <%
18         // 3. SCRIPTLET (<% %>): Usiamo un ciclo for per la logica
19         for(int i = 1; i <= 3; i++) {
20     %>
21     <li>Iterazione numero: <%= i %></li>
22     <%-- Un'altra EXPRESSION per stampare l'indice del ciclo --%>
23     <%
24         } // Chiusura dello scriptlet del ciclo
25     %>
26     </ul>
27 </body>
28 </html>

```

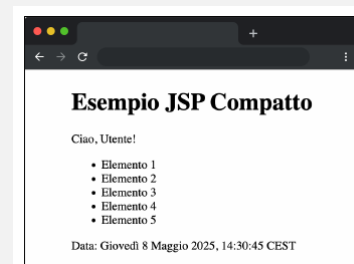
15.3 Esempio

Esempio JSP e Anteprima

```

1  <%@ page language="java" contentType="text/html;
   charset=UTF-8" %>
2  <!DOCTYPE html>
3  <html>
4  <body>
5      <h1>Esempio JSP Compatto</h1>
6      <%
7          String nome = "Utente";
8          int numero = 5;
9      %>
10     <p>Ciao, <%= nome %>!</p>
11 </body>
12 </html>

```



15.4 Oggetti impliciti

Per facilitare la scrittura del codice, l'ambiente JSP mette automaticamente a disposizione dello sviluppatore 9 oggetti impliciti senza che ci sia bisogno di istanziarli automaticamente.

- **request**: Rappresenta la richiesta HTTP inviata dal client.
Si usa per recuperare parametri del form, header e informazioni sul browser dell'utente.
- **response**: Rappresenta la risposta HTTP inviata al client.
Serve per impostare header, aggiungere cookie o effettuare reindirizzamenti (redirect).
- **out**: È l'oggetto (JspWriter) utilizzato per scrivere il contenuto nel corpo della risposta che verrà inviata al browser.
- **page**: Rappresenta l'istanza corrente della Servlet generata dalla pagina JSP (equivale al `this` in Java).

- **pageContext**: Fornisce l'accesso a tutti i contesti della pagina e permette di gestire gli attributi in vari ambiti (scope).
- **session**: Rappresenta la sessione dell'utente.
Permette di memorizzare informazioni che devono persistere mentre l'utente naviga tra diverse pagine del sito.
- **application**: Rappresenta l'intera applicazione web (ServletContext).
I dati salvati qui sono condivisi da tutti gli utenti e da tutte le pagine.
- **config**: Rappresenta la configurazione della Servlet (ServletConfig) e permette di leggere eventuali parametri di inizializzazione definiti nel file `web.xml`.
- **exception**: Disponibile solo se la pagina è definita come "pagina di errore".
Contiene l'eccezione che ha causato il malfunzionamento.

15.4.1 Scope (visibilità)

Ognuno di questi oggetti, così come qualsiasi variabile create nell'applicazione, possiede uno "scope" (ambito).

Lo scope è fondamentale perché determina il ciclo di vita e il livello di visibilità dell'oggetto nella memoria del server.

Esistono 4 livelli di scope:

- **Page**: È il livello base.
L'oggetto esiste ed è visibile solo all'interno della specifica pagina JSP che lo ha creato.
- **Request**: L'oggetto vive per il tempo necessario a elaborare una singola richiesta HTTP.
È utile se la richiesta viene passata tra più componenti.
- **Session**: L'oggetto è legato alla sessione del singolo utente.
I dati salvati qui rimangono in memoria e sono condivisi tra tutte le pagine visitate da quell'utente specifico.
- **Application**: È il livello massimo.
L'oggetto è condiviso in modo globale tra tutti gli utenti e tutte le pagine dell'intera applicazione web.

15.5 JavaBean

L'uso eccessivo di Scriptlet rende le pagine JSP caotiche e impossibili da mantenere per i web designer, per risolvere questo problema si utilizzano i JavaBean.

Un JavaBean è una semplicissima classe Java (POJO) che funge da contenitore di dati e logica, e deve rispettare regole rigide:

- deve essere pubblica
- deve avere costruttore senza argomenti
- deve avere attributi privati e metodi pubblici get e set per accedere a quegli attributi

Invece di scrivere codice Java complesso, la JSP utilizza delle speciali "Azioni" per interagire con il Bean in modo pulito:

- **<jsp:useBean>**: Cerca un Bean in un determinato scope o lo istanzia se non esiste.
- **<jsp:getProperty>** e **<jsp:setProperty>**: Leggono o scrivono le proprietà del Bean.

Un aspetto molto potente è l'uso di `property="*"`.

Quando viene usata, il server legge in automatico tutti i parametri in arrivo da un form HTML e invoca i rispettivi metodi set del Bean convertendo i tipi di dato, eliminando completamente la necessità di leggere e convertire manualmente i dati della richiesta.

15.6 Declino

Nonostante i tentativi di incapsulare la logica (come i JavaBean e il pattern MVC), il modello basato su JSP è considerato oggi superato a causa di intrinseci **limiti architetturali** ¹⁹.

I motivi principali del suo declino sono:

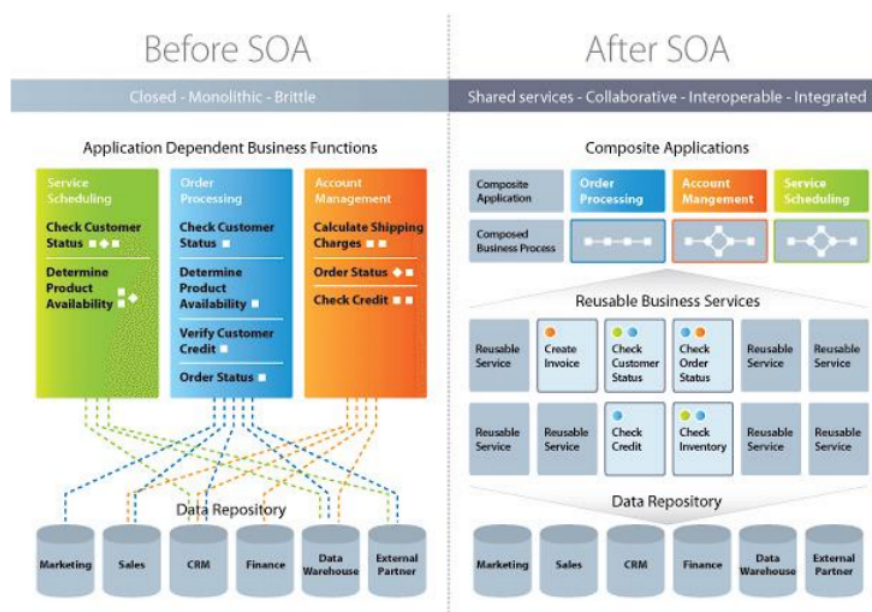
- **Forte accoppiamento (Spaghetti Code)**: L'inserimento di frammenti Java (scriptlet) dentro l'HTML rende il codice molto difficile da leggere, testare e mantenere ¹⁹.
- **Sviluppo lento**: Poiché le JSP devono essere tradotte in Servlet e compilate dal server, il ciclo di sviluppo e testing risulta molto più lento rispetto a tecnologie moderne ¹⁹.
- **Evoluzione del Web**: Le JSP sono nate nel 1999 per generare pagine interamente sul server ²⁰. Oggi, il paradigma dominante si è spostato verso le **Single Page Application (SPA come React, Angular, Vue)**, dove il browser si occupa di costruire la View dinamicamente (client-side), mentre il server funge solo da fornitore di dati puri (JSON) tramite **servizi REST** ²⁰.
- **Alternative Server-Side**: Anche quando serve generare l'HTML sul server, oggi si preferiscono **motori di template moderni come Thymeleaf**, che garantiscono una separazione molto più rigida tra il codice HTML e la logica Java, oppure framework per il Server-Side Rendering ibrido (come Next.js o Nuxt) ²⁰.

Capitolo 16

SOA - Software Oriented Architecture

16.1 Definizione

SOA (Architettura orientata ai servizi) è **uno stile architettonico**, che si concentra su elementi discreti riutilizzabili chiamati **servizi**, invece che su un design monolitico, per costruire le applicazioni.



16.2 Elementi fondamentali di SOA

16.2.1 Componenti

- **Servizio:** unità funzionale autonoma che esegue una funzione specifica.
- **Descrizione del servizio:** documento che definisce l'interfaccia, le operazioni e i parametri di un servizio.

16.2.2 Ruoli

- **Service Registry:** catalogo che conserva le descrizioni dei servizi, gli endpoint(URL) e le informazioni di binding (protocolli supportati, formati dei dati).

- **Service Provider**: chi crea un servizio e ne pubblica la descrizione nel registry.
- **Service Requestor**: chi consulta il registry per trovare un servizio adatto e lo invoca.

16.3 Servizi

16.3.1 Definizione Informale

Un servizio è un'entità software **indipendente** che può essere scoperta e invocata da altri sistemi software attraverso una rete.

16.3.2 Definizione Formale

Un servizio Web è un'**applicazione software**

- Identificata da un **URI**.
- Ha interfacce in grado di essere definite, descritte e scoperte.
- Supporta **interazioni dirette** con altri sistemi software per mezzo di messa e protocolli basati su internet

16.3.3 Cosa fa un servizio?

Ogni servizio deve:

- Fornire le proprie funzionalità ai richiedenti (che possono essere altri servizi).
- Descrivere le sue funzionalità da scoprire e selezionare.
- Essere accessibile attraverso l'uso di protocolli rete noti.
- Supportare la collaborazione con altri servizi per fornire soluzioni complesse.
- Rispondere alle esigenze aziendali dei client e ai requisiti del dominio.
- Garantire un livello di qualità di servizio.

16.3.4 Composizione dei servi

Una **composizione** è un insieme di servizi interconnessi che, presi nel loro insieme, costituiscono a loro volta un nuovo servizio, riutilizzabile in altre composizioni.

Due servizi **sono interconnessi** quando almeno uno dei 2 richiama la funzionalità esposta dall'altro attraverso la sua interfaccia (endpoint, API).

Perché la composizione funzioni, è necessaria la **compatibilità** tra i servizi: i servizi devono parlare la stessa lingua sia **sintatticamente** (stessi formati di messaggio, stessi tipi di stato) sia **semanticamente** (stesso significato dei dati scambiati).

Per questo ogni provider deve esporre **un'interfaccia pubblicata**, descritta in modo formale e accessibile a chi vuole consumarla.

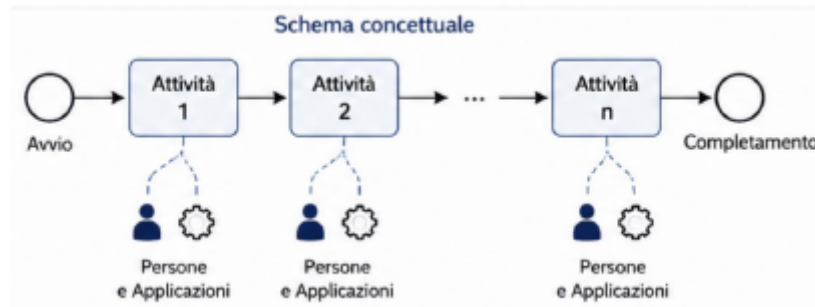
Quando una composizione esprime un **flusso di lavoro** coordinato con un ordine di esecuzione, condizioni e dipendenze tra i servizi parliamo di **processo di business**.

16.4 Business Process

16.4.1 Definizione

Un **business process** è un workflow strutturato di attività correlate, eseguite da **persone e applicazioni**, con l'obiettivo di ottenere un **risultato ben definito**.

I processi vengono realizzati tramite **composizione dei servizi**, in cui ogni attività è implementata come un servizio software coordinato all'interno di un flusso.

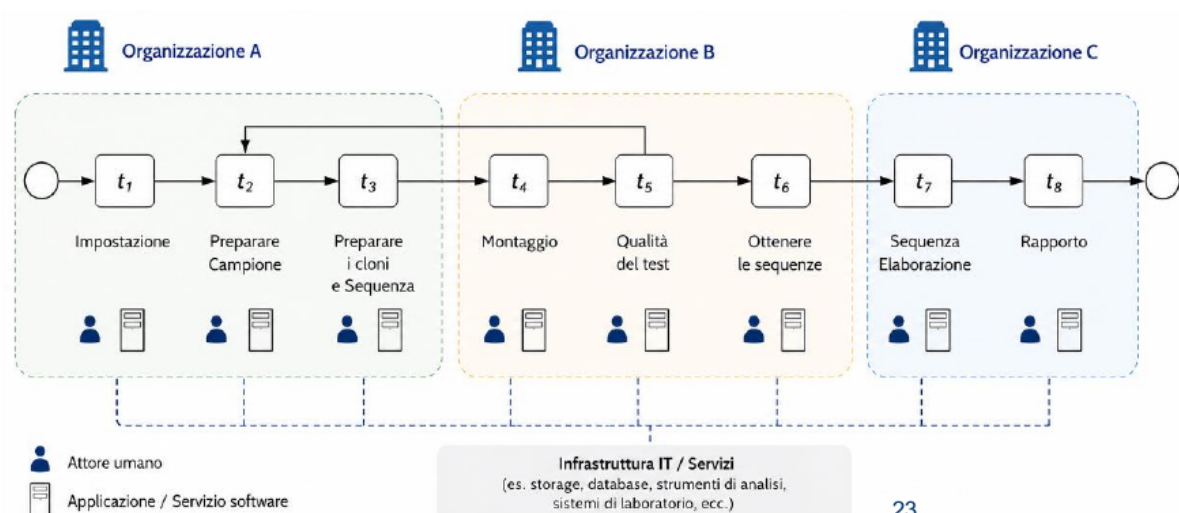


16.4.2 Caratteristiche

- Può contenere condizioni definite che ne determinano l'avvio e uscite definite al suo completamento.
- Può comportare interazioni formali o relativamente informali tra i partecipanti (umani e software).
- Può contenere una serie di attività automatizzate e/o manuali.
- È distribuito e personalizzato al di là dei confini all'interno delle organizzazioni e tra di esse, e spesso si estende a più applicazioni con piattaforme tecnologiche diverse.
- Ha una durata che può variare notevolmente.
- Di solito è di lunga durata.

Una singola istanza può durare mesi o addirittura anni.

16.4.3 Esempio



16.4.4 Orchestrazione

Definizione

Descrive come i servizi interagiscono tra loro, governati dalla **logica di business** e da un preciso **ordine di esecuzione** dal punto di vista e sotto il controllo di un singolo attore (servizio orchestratore).

Caratteristiche principali

- **Controllo centralizzato**: Un servizio orchestratore decide cosa succede e quando.
- **Gesione esplicita**: L'orchestratore conosce tutti i dettagli del flusso.

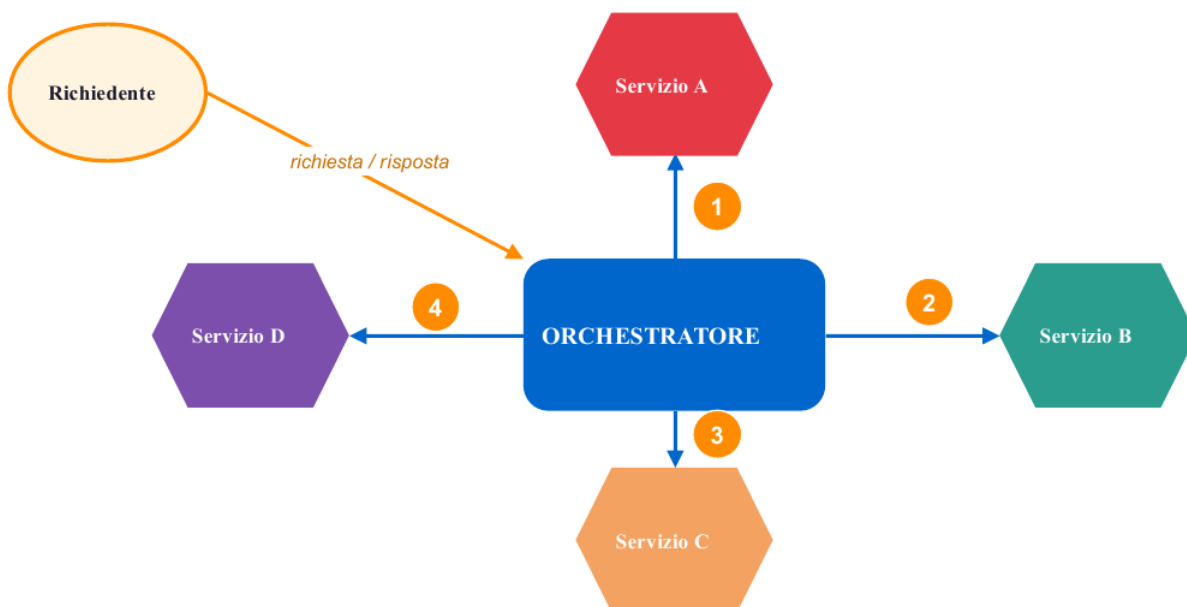
Vantaggi

- Monitoraggio semplificato (tutto passa dall'orchestratore)
- Più facile gestire errori e transazioni

Svantaggi

- Dipendenza dall'orchestratore.
- Accoppiamento più stretto tra servizi.
- Scalabilità limitata.

Schema



L'orchestratore conosce e governa il flusso: invoca i servizi nel giusto ordine, riceve le risposte, decide cosa fare dopo. Il controllo è centralizzato, i servizi non si parlano direttamente.

16.4.5 Coreografia

Definizione

Descrive la sequenza di interazioni tra più parti coinvolte nel processo **dal punto di vista di tutte le parti**.

Definisce lo stato condiviso delle interazioni tra le entità.

Ogni servizio è responsabile di portare a termine il proprio compito, invocando anche altri servizi.

Di solito viene implementato utilizzando eventi.

Caratteristiche principali

- **Controllo distribuito**: Ogni servizio agisce autonomamente.
- **Comunicazione tramite eventi**: Spesso i servizi reagiscono a eventi pubblicati da altri

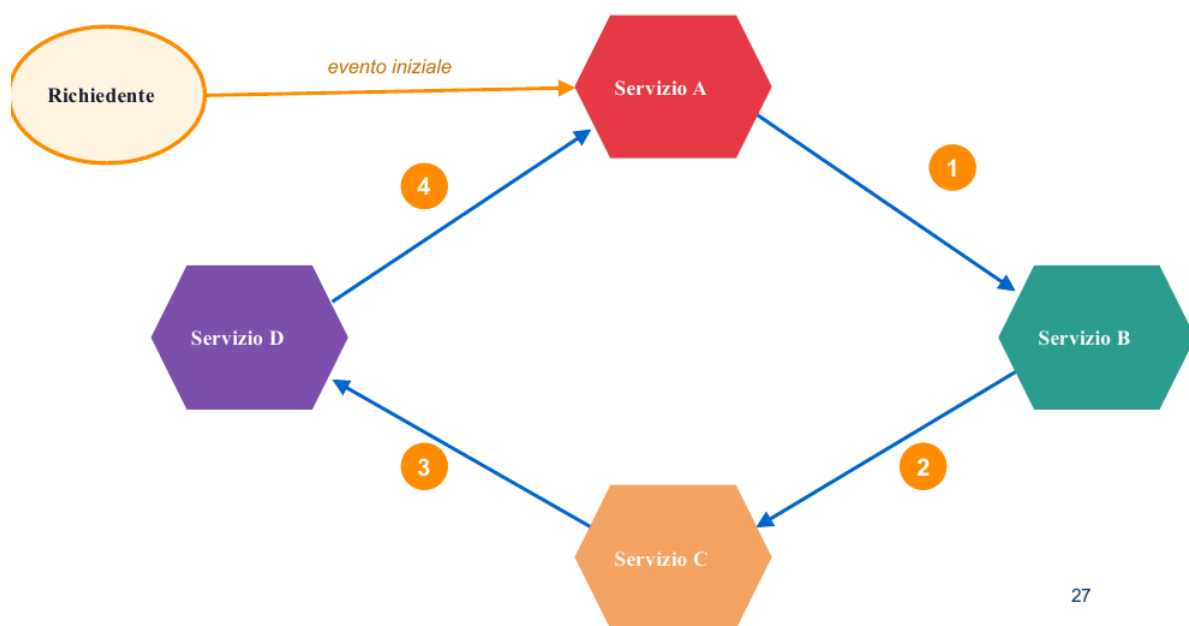
Vantaggi

- Alta scalabilità (ogni servizio è indipendente).
- Basso accoppiamento tra servizi.
- Resilienza (nessun single point of failure).

Svantaggi

- Difficile da monitorare (flusso distribuito).
- Gestione errori complessa.
- Testing end-to-end più difficile.

Schema



27

Nessun controllore centrale: ogni servizio sa cosa fare quando riceve un evento e a chi notificare il proprio. Il flusso emerge dalle interazioni; il coordinamento è distribuito.

Soste

16.5 Web API

16.5.1 Definizione

Le Web API sono interfacce che permettono la comunicazione tra applicazioni software attraverso il web seguendo i principi come componenti indipendenti.

16.5.2 Caratteristiche

1. Interfaccia ben nota

- Spesso supportata da un linguaggio di descrizione standard (WSDL o OpenAPI).
- Autoesplicativa: Include endpoints, metodi, parametri e risposte.
- Contratto di servizio: Definisce chiaramente le interazioni possibili.
- Possibile gestione automatica da parte del middleware.

2. Punto di accesso unico

- Indirizzabile tramite URI base:
Il servizio ha un endpoint principale univoco.
- Scopribile: Attraverso registri di servizi o discovery services.
- Gateway unificato: Singolo punto di ingresso per tutte le funzionalità di servizio.

3. Scambio di dati basato su documenti

- Utilizzo di un formato di rappresentazione standard.
- Usually stateless: Ogni richiesta contiene tutte le informazioni necessarie.

Esempio: Risposta JSON da OpenWeatherMap API

```
1 {  
2   "temp": 22.5,  
3   "feels_like": 21.8,  
4   "humidity": 65,  
5   "wind_speed": 5.2  
6 }
```

16.5.3 Esempio: openweathermap

Un semplice esempio di servizio web è un'API meteo.

- **Identificato da un URI:**

L'API meteo è accessibile tramite un URL.

- **Definita, descritta e scoperta:**

L'API fornisce una documentazione che descrive le sue funzionalità, i parametri di input (come il nome della città o le coordinate) e gli output previsti (come la temperatura o le condizioni meteorologiche).

- **Supporta le interazioni attraverso i messaggi dei protocolli internet:**

Le applicazioni comunicano con l'API meteo tramite HTTP o HTTPS.

Ad esempio, un'applicazione può inviare una richiesta GET per recuperare i dati meteo attuali.

Esempio di interazione

- **Client Request:** `https://api.openweathermap.org/data/2.5/weather?q=Berlin&appid=your_api_key`
- **Server Response:** Dati in formato JSON contenuti informazioni meteo per Berlino, come ad esempio

```

1 {
2   "weather": [{"description": "clear sky"}],
3   "main": {"temp": 286.15},
4   "name": "Berlin"
5 }
```

16.6 Operazioni di base in SOA

L'interazione tra service provider, requestor e registry si articola in 3 operazioni fondamentali:

- **Publish**

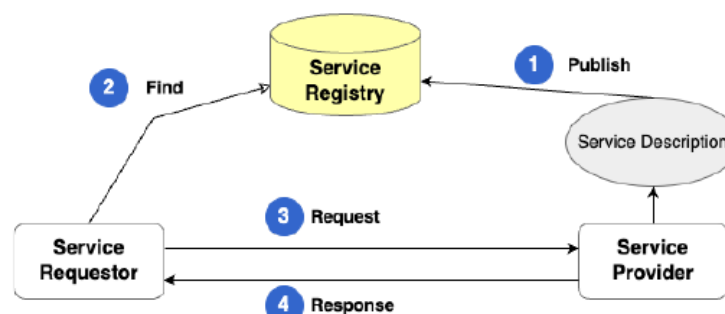
Il provider pubblica nel registry la descrizione del proprio servizio (operazioni esposte, formati di input/output, endpoint).

- **Find**

Il requestor interroga il registry alla ricerca di un servizio compatibile con le proprie esigenze, e ne ottiene la descrizione e l'endpoint.

- **Bind/Invoke**

Il requestor stabilisce una connessione diretta con il provider e invoca il servizio (richiesta-risposta).



Il registry partecipa solo alle prime 2 fasi: una volta ottenuto l'endpoint, requestor e provider comunicano direttamente.

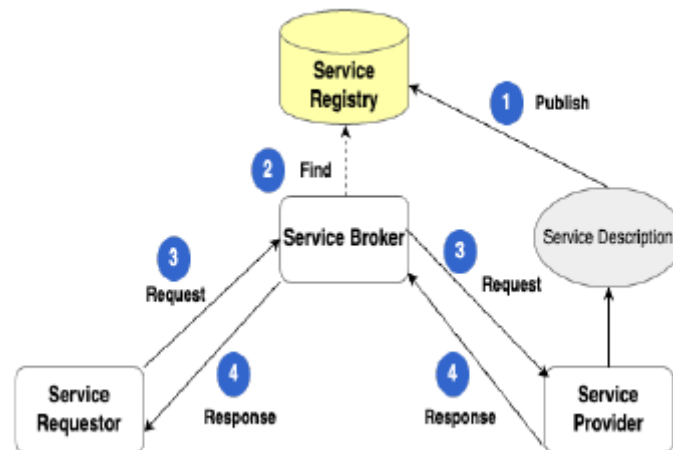
È un ruolo passivo da catalogo.

16.7 Comunicazione basata su broker

16.7.1 Definizione

Nel modello classico il registry è passivo: indicizza i servizi ma non partecipa all'invocazione.

Quando ai requirements si aggiungono **sicurezza, composizione, mediazione di protocollo o caching** il registry evolve in **Service Broker**: un intermediario attivo che si interpone in ogni interazione.



16.7.2 Cosa cambia rispetto al registry?

- Le richieste passano **attraverso il broker**, non più direttamente al provider.
- Il broker può **autenticare, instradare, convertire** formati e protocolli, comporre più servizi in un'unica chiamata.
- Il **binding diventa dinamico**: il broker sceglie l'istanza del provider al momento della richiesta.

16.7.3 Vantaggi

- **Disaccoppiamento**

Provider e requestor non si conoscono direttamente, un provider può essere aggiornato senza impatto sui consumatori.

- **Interoperabilità**

La mediazione permette a sistemi con protocolli diversi di dialogare.

- **Scalabilità**

Il broker distribuisce il carico fra più istanze dello stesso servizio.

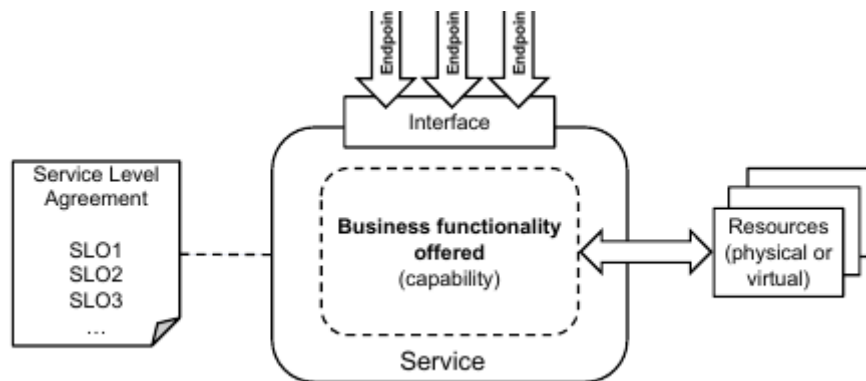
16.8 SLA - Service Level Agreement

16.8.1 Definizione informale

Lo SLA definisce le modalità di erogazione di un servizio piuttosto che limitarsi a descrivere ciò che il servizio comporta.

16.8.2 Definizione formale

Un SLA è un contratto formalizzato che delinea gli standard, le aspettative e le metriche di prestazione concordate tra un fornitore di servizi e un consumatore di servizi.



16.8.3 Esempi di metriche SLA

Metric	Definition	Example Target
Availability	Percentage of time a service is operational.	99.9% per month.
Response Time	Time to acknowledge a reported issue.	15 minutes.
Resolution Time	Time to resolve a critical incident.	4 hours.
Throughput	Volume of transactions processed per second.	1,000 requests/sec.
Error Rate	Percentage of failed transactions.	<1% per month.

16.9 Vantaggi e Svantaggi

16.9.1 Vantaggi

- Riutilizzabilità.
- Processo di sviluppo agile e orientato al business.
- Economia dei servizi.
- Scalabilità.
- Ottimizzazione e riduzione dei costi.

16.9.2 Svantaggi

- Gestione del ciclo di vita complesso.
- Dependency Hell.
- Integrazione con le soluzioni legacy.

Capitolo 17

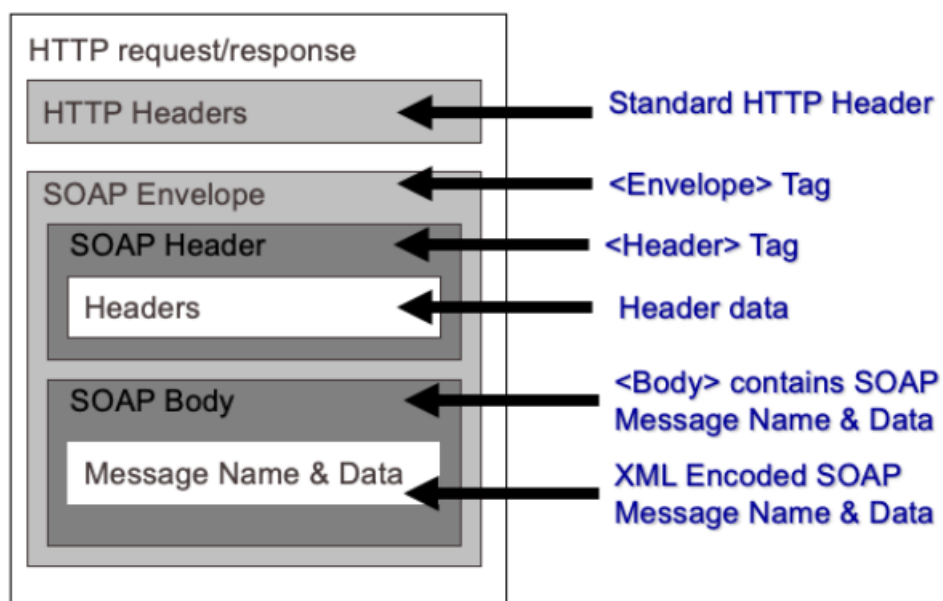
SOAP - Simple Object Access Protocol

17.1 Definizione

SOAP è un **protocollo di scambio messaggi basato su XML** che consente ad applicazioni software di comunicazione utilizzando messaggi XML.

- Gli spazi dei nomi XML sono utilizzati per fornire una semantica ai dati (non solo ai tipi di dati)
- È agnostico rispetto al sottosistema di trasporto
- Permette di creare messaggi di richiesta e risposta in diverse configurazioni.
- Non è legato ad alcun linguaggio o piattaforma di programmazione (nemmeno ai servizi web)
- Utilizzato nei servizi orientati ai documenti

17.2 Componenti di un messaggio SOAP



- **SOAP Envelope**
 - Ingloba il contenuto del messaggio.
- **SOAP Header (opzionale)**
 - Maggiore flessibilità, può essere elaborato dai nodi tra l'origine e la destinazione.
 - Contiene blocchi di informazioni su come elaborare il messaggio:
 - * Impostazione di instradamento e consegna.
 - * Asserzioni di autenticazione/autorizzazione.
 - * Contesti di transazione
- **SOAP Body**
 - Messaggio effettivo da consegnare ed elaborare.
 - Sia per le informazioni di richiesta che per quelle di risposta.

17.3 SOAP con HTTP

Sebbene SOAP possa utilizzare diversi protocolli di trasporto, HTTP è quello più comunemente utilizzato.

- Le richieste HTTP sono costituite da un metodo, come POST o GET, seguito dall'URL richiesto e dalla versione del protocollo.
- Le risposte seguono la semantica di HTTP per fornire il codice di stato della risposta.

17.3.1 Modelli di scambio messaggi via HTTP

Esistono due modelli per lo scambio di messaggi SOAP via HTTP:

- Il modello **SOAP request-response** in cui il metodo POST viene utilizzato per portare i messaggi SOAP nel corpo delle richieste/risposte HTTP.

Esempio di request-response

```

1 POST /Reservations HTTP/1.1
2 Host: travelcompany.example.org
3 Content-Type: application/soap+xml; charset="utf-8"
4 Content-Length: 230
5
6 <?xml version='1.0' ?>
7 <env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope">
8   <env:Header> ... </env:Header>
9   <env:Body> ... </env:Body>
10 </env:Envelope>

```

Listing 17.1: Richiesta HTTP

```

1 HTTP/1.1 200 OK
2 Content-Type: application/soap+xml; charset="utf-8"
3 Content-Length: 200

```

```

4 <?xml version='1.0' ?>
5 <env:Envelope
6   xmlns:env="http://www.w3.org/2003/05/soap-envelope">
7   <env:Header>...</env:Header>
8   <env:Body>...</env:Body>
9 </env:Envelope>

```

Listing 17.2: Risposta HTTP

- Il modello **SOAP response** in cui nelle richieste HTTP viene utilizzato il metodo GET per ottenere il messaggio SOAP nel corpo della risposta.

Esempio di response

```

1 GET /travel.example.org/reservations?code=F3T HTTP/1.1
2 Host: travelcompany.example.org
3 Accept: text/html;q=0.5, application/soap+xml

```

Listing 17.3: Richiesta HTTP

```

1 HTTP/1.1 200 OK
2 Content-Type: application/soap+xml; charset="utf-8"
3 Content-Length: 200
4 <?xml version='1.0' ?>
5 <env:Envelope
6   xmlns:env="http://www.w3.org/2003/05/soap-envelope">
7   <env:Header>...</env:Header>
8   <env:Body>...</env:Body>
9 </env:Envelope>

```

Listing 17.4: Risposta HTTP

Utilizzando SOAP con HTTP, bisogna ricordarsi di impostare l'intestazione **Content-Type** come **application/SOAP+xml**

17.4 WSDL

17.4.1 Definizione

WSDL è un linguaggio di descrizione dei servizi web basato su XML.

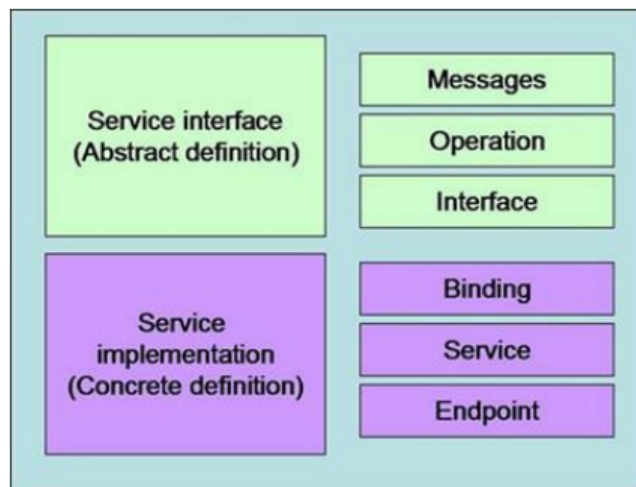
17.4.2 Cosa descrive un documento WSDL

WSDL consente di descrivere quattro dati principali per un servizio:

- Informazioni sull'**interfaccia** che descrivono tutte le operazioni pubblicamente disponibili di un servizio.
- Dichiarazioni di **tipi di dati** per tutti i messaggi.
I tipi complessi possono essere dichiarati (utilizzando SOAP) e utilizzati.
- Informazioni di **binding** sul protocollo di trasporto.

- Informazioni sull'**indirizzo** per la localizzazione del servizio

17.4.3 Modello concettuale



- Nella parte **astratta**
 - Descrizione di un servizio in termini di **messaggi** che invia e riceve attraverso un sistema di tipi, tipicamente W3C XML Schema
 - Un'**operazione** associa modelli di scambio di messaggi a uno o più messaggi.
 - **I modelli di scambio di messaggi** definiscono la sequenza e la cardinalità dei messaggi scambiati tra i nodi (servizi).
 - Un'**interfaccia** raggruppa queste operazioni in modo indipendente del canale di comunicazione.
- Nella parte **concreta**
 - I **binding** specificano il protocollo di trasporto per le interfacce.
 - Un **endpoint** associa un URI a un binding.
 - Un **servizio** raggruppa gli endpoint che implementano un'interfaccia comune.

17.4.4 Definizione del binding

- **Binding:**
 - L'attributo **name** definisce il nome dei binding.
Con questo nome si può fare riferimento ad esso quando si definisce un endpoint di servizio.
 - Ogni nome di binding deve essere univoco all'interno dello spazio dei nomi del target WSDL 2.0.
 - L'attributo **interface** contiene il nome di una delle interfacce definite.
 - L'attributo **type** definisce il formato del messaggio da utilizzare.
In questo caso è SOAP.

- **wssoap:protocol** definisce il protocollo di "trasporto".

Nell'esempio, i messaggi soap verranno trasportati utilizzando HTTP.

- **Operation:**

- L'attributo **ref** dell'elemento operation fa riferimento a una specifica operazione (già definita nella sezione interface).
- L'attributo **wssoap:code** dell'elemento fault definisce il codice di errore che determina l'invio di questo messaggio.

- **Fault:**

- L'attributo **ref** definisce quale errore (già definito nella sezione dell'interfaccia) ci si riferisce per questo binding.
- L'attributo **wssoap:code** dell'elemento fault definisce il codice di errore che determina l'invio di questo messaggio.

```
1 <binding name = "myServiceInterfacesOAPBinding"
2   interface = "tns:myServiceInterface"
3   type = "http://www.w3.org/ns/wsd1/soap"
4   wssoap:protocol = "http://www.w3.org/2003/05/soap/bindings/HTTP/">
5   <operation ref = "tns:checkServiceStatusOp"
6     wssoap:mep = "http://www.w3.org/2003/05/soap/mep/soap-response/" />
7   <fault ref = "tns:dataFault" wssoap:code = "soap:Sender" />
8 </binding>
```

17.4.5 Definizione di service/endpoint

- **Service:**

- L'attributo **name** definisce il nome del servizio.

Ogni nome di servizio deve essere unico all'interno dello spazio dei nomi di destinazione.

- L'attributo **interface** definisce il nome dell'interfaccia (precedentemente definita nella sezione interface) implementata dal servizio.

- **Endpoint:**

- L'attributo **name** definisce il nome dell'endpoint, che deve essere unico all'interno dei servizi.
- L'attributo **binding** specifica quale binding già definito verrà utilizzato da questo endpoint.
- L'attributo **address** specifica l'indirizzo fisico dove il servizio sarà disponibile.

Esempio di service/endpoint

```
1 <service name = "myService" interface = "tns: myServiceInterface">
2   <endpoint name = "myServiceEndpoint"
3     binding = "tns:myServiceInterfaceSOAPBinding"
4     address = "http://yoursite.com/MyService" />
```

```
5 </service>
```

17.4.6 Declino di SOAP

L'obiettivo principale di SOAP era creare uno standard universale tra le applicazioni che fosse indipendente da piattaforme, linguaggi e protocolli di trasporto.

Fattori di declino

- **Complessità eccessiva:**
 - Struttura XML verbosa e rigida.
 - Curve di apprendimento ripide.
 - Costi di implementazione elevati
- **Performance insufficienti:**
 - Overhead significativo nella trasmissione dei dati.
 - Parsing XML computazionale costoso.
 - Latenza più elevata rispetto alle alternative
- **Ascesa delle alternative:**
 - REST: Più semplice, leggibile e veloce.
 - JSON: Formato dati leggero e più facile da elaborare.

Capitolo 18

REST - Representational State Transfer

18.1 Definizione

REST è uno **stile architettónico** per i sistemi distribuiti.

- Le risorse sono **identificate da URI**.
- Le risorse vengono manipolate attraverso le loro **rappresentazioni** (possono esistere più rappresentazioni di una risorsa).
- Comunicazione tramite messaggi che sono **autodescrittivi e senza stato**.
- Lo stato dell'applicazione evolve per mezzo della **manipolazione** delle risorse da parte dei client.
- Hypermedia è il modo che il server ha per **guidare il comportamento del client**.

18.2 Risorse e Interfaccia uniforme

18.2.1 Risorse

Una risorsa è una qualsiasi informazione che può essere nominata.

- Le risorse di solito cambiano con l'interazione con il client.
- Le risorse hanno **identificatori**
es: `unimib.it/{matricola}/pianoStudio`, `{matricola}` è chiamato path parameter.

18.2.2 Interfaccia uniforme

RESTful prevede quattro operazioni HTTP

Operazione	Descrizione
PUT	Modifica una risorsa / Nuovo stato
POST	Crea una nuova risorsa
GET	Recupera lo stato di una risorsa
DELETE	Elimina una risorsa

I messaggi REST sono completamente auto-descrittivi, contengono tutti i metadati necessari per la loro interpretazione, senza fare affidamento su contesto di sessione esterno.

L'esecuzione è stateless, dopo ogni operazione il componente dimentica tutto del chiamante, rendendo le operazioni idempotenti e semplificando la scalabilità del sistema.

18.3 Statelessness

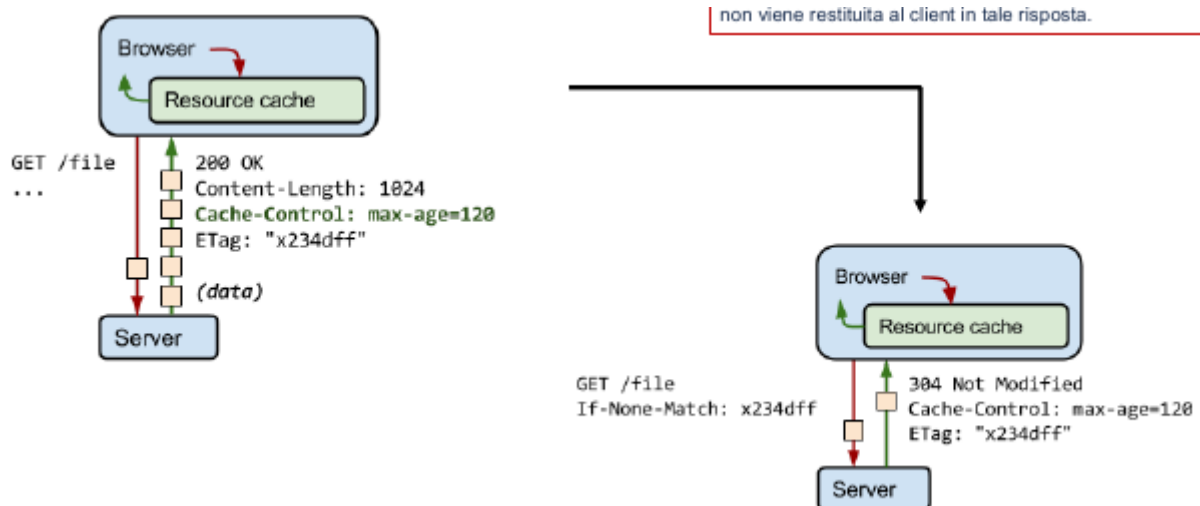
- Ogni richiesta del client al server deve contenere tutte le informazioni necessarie a comprendere la richiesta e non può sfruttare alcun contesto memorizzato sul server.
- La mancanza di stato richiede **messaggi autodescrittivi**.

18.4 Cacheability

La Cacheability è una proprietà fondamentale delle architettura REST e indica la **capacità delle risposte HTTP di essere memorizzate in cache** da client, proxy, gateway o CDN, al fine di migliorare l'efficienza, ridurre il carico sui server e diminuire la latenza percepita.

La cacheability richiede un sistema a livelli:

- Almeno un livello di cache.
- Nella cache è memorizzata l'ultima rappresentazione di una risorsa.
- Una cache viene invalidata quando viene modificato lo stato della risorsa (PUT, DELETE).
- Vengono memorizzati nella cache solo le risposte ai GET.



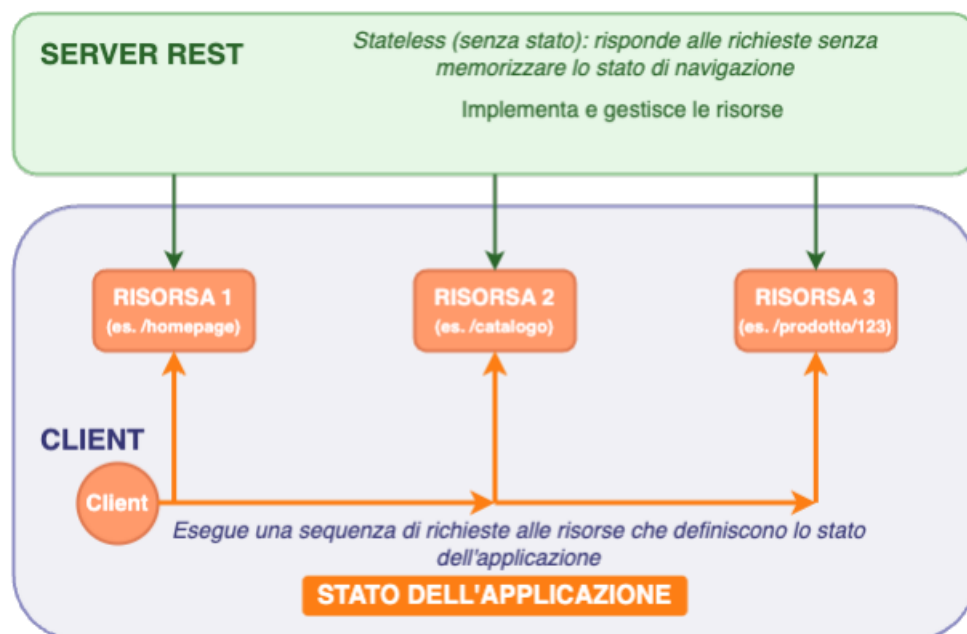
18.4.1 Header utilizzati per il cache-control

Intestazione	Proprietà
Expires	Data HTTP. Mantenere in cache fino alla scadenza
Cache-Control: max-age	Un secondi. Mantenere in cache per il tempo indicato
Cache-Control: s-maxage	Secondi. Come sopra, ma solo per i proxy
Cache-Control: public	La risorsa può essere salvata in una cache condivisa
Cache-Control: no-cache	La risorsa può essere memorizzata nella cache, ma deve essere convalidata dal server di origine prima di ogni riutilizzo
Cache-Control: no-store	Non memorizzare nella cache
Cache-Control: must-revalidate	La risorsa può essere memorizzata in cache e riutilizzata quando è "fresca". Se diventa stantia (stale), deve essere convalidata con il server di origine prima di essere riutilizzata.
Cache-Control: proxy-revalidate	Come sopra, ma solo per i proxy

18.5 Stato dell'applicazione

Nel contesto REST, lo **stato dell'applicazione** rappresenta il "percorso" del client nel sistema: la sequenza di risorse e interazioni attraversate durante l'uso dell'applicazione.

- È **gestito dal client**.
- Include le risorse richieste, link seguiti, dati ricevuti e modificati.
- Si evolve man mano che il client invia richieste e riceve risposte che includono link e altre risorse.



18.5.1 Hypermedia control

I client devono seguirne i link forniti dal server, i link devono essere "classificati" con un significato semantico in modo che un client intelligente sappia a cosa punta un link.

18.5.2 Esempio : Gestione del curriculum universitario

- **GET `unimib.it/{matricola}`** : restituisce informazioni sullo studente e l'elenco di altre risorse collegate:
 - nome, cognome, data-di-nascita, etc.
 - `unimib.it/{matricola}/badges`
 - `unimib.it/{matricola}/pianostudio`
 - `unimib.it/{matricola}/storicorate`
- **GET `unimib.it/{matricola}/pianostudio`** :
 - **GET** per ottenere l'elenco degli esami a cui lo studente è iscritto,
 - **POST** per iscriversi a un esame,
- **GET `unimib.it/{matricola}/pianostudio/{codice}`** :
 - **GET** per ottenere informazioni sull'esame a cui lo studente è iscritto
 - **PUT** per modificare la registrazione dell'esame,
 - **DELETE** per eliminare la registrazione dell'esame

```
1 HTTP/1.1 200 OK
2 Content-Type: application/vnd.unimib.student+json
3 Content-Length: ...
4
5 {
6   "studente": {
7     "matricola": "123456",
8     "nome": "Mario",
9     "cognome": "Rossi",
10    "data_di_nascita": "2000-04-15",
11    "email": "mario.rossi@unimib.it",
12    "_links": {
13      "self": {
14        "href": "unimib.it/123456"
15      },
16      "badges": {
17        "href": "unimib.it/123456/badges"
18      },
19      "piano_studio": {
20        "href": "unimib.it/123456/pianostudio"
21      },
22      "storico_rate": {
23        "href": "unimib.it/123456/storicorate"
24      }
25    }
26  }
27 }
```

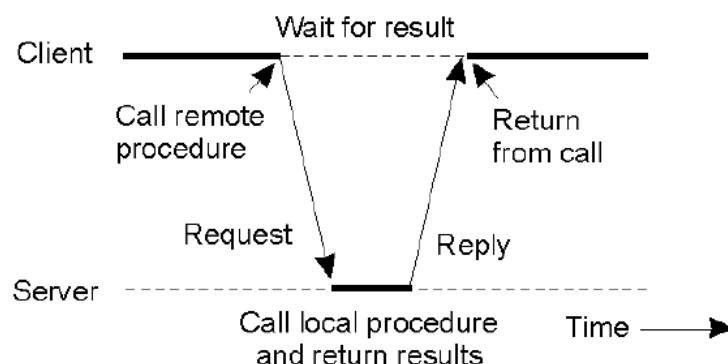
Capitolo 19

RPC - Remote Procedure Call

19.1 Definizione

L'RPC è un meccanismo progettato per permettere ai programmi di invocare procedure (funzioni o metodi) **che si trovano fisicamente su macchine remote**.

A differenza dei tradizionali approcci basati sullo scambio esplicito di messaggi dove il programmatore deve gestire e scrivere attivamente il codice per la comunicazione di rete, **l'obiettivo fondamentale di RPC** è garantire la trasparenza, questo significa che il programmatore effettua la chiamata nel codice esattamente nel solito modo in cui chiamerebbe una funzione locale, nascondendo quasi del tutto l'esistenza di una rete.



19.2 Architettura RPC

Lo scopo principale dell'Architettura RPC è mascherare la comunicazione di rete e far apparire l'invocazione di una funzione remota esattamente come se fosse una chiamata locale.

Per ottenere questa "trasparenza", l'architettura RPC delega tutta la complessità della comunicazione a un middleware composto da due elementi speculari, chiamati **stub** (o proxy/skeleton)

19.2.1 Client stub (o proxy)

È un oggetto che risiede sulla macchina del client e che offre al programma chiamante la stessa identica interfaccia della procedura remota, quando il programma effettua la chiamata, in realtà sta invocando il client stub.

Il compito di questo componente è raccogliere i parametri forniti, impacchettarli in un messaggio, spedire la richiesta attraverso la rete e sospendere l'esecuzione in attesa.

Quando arriva la risposta del server, estrae il risultato e lo restituisce al chiamante, simulando la presenza locale della risorsa.

19.2.2 Server Stub (o skeleton)

Risiede sulla macchina del server.

Il suo ruolo è rimanere in ascolto, ricevere il messaggio di rete inviato dal client, decompacchettarlo per estrarre i parametri originali e invocare la vera procedura locale.

Una volta terminata l'elaborazione, il server stub raccoglie il risultato, crea un messaggio di risposta e lo invia indietro al client.

19.3 Procedura generale e il ruolo del Middleware

Quando un programma sul client invoca una procedura remota, la sua esecuzione viene momentaneamente bloccata.

La chiamata viene intercettata da un software intermedio, chiamato **middleware** che si occupa di tradurre la chiamata locale in un pacchetto di richiesta, lo spedisce al server e rimane in attesa.

Quando il server remoto termina l'esecuzione e restituisce la risposta, il middleware lo scompatta e la consegna al programma chiamante, che a quel punto sblocca la sua esecuzione e procede.

In questo modo l'RPC riesce a "**mascherare**" il modello client-server.

19.3.1 Vantaggi e Svantaggi

Vantaggi

È molto intuitiva perchè utilizza una semantica bene nota a tutti i programmari e si adatta perfettamente alle architetture clien-server.

Svantaggi

Poichè la procedura viene eseguita su una macchina diversa da quella chiamante, i due processi vivono in **spazi di indirizzamento della memoria differenti**.

Questo rende il passaggio dei parametri e dei risultati molto complesso da gestire, soprattutto perchè gli indirizzi di memoria hanno valenza puramente locale e non hanno alcun senso se spediti a un computer diverso.

19.4 Passaggio dei parametri

Il problema del passaggio dei parametri in una RPC nasce dal fatto che il processo chiamante (client) e il processo chiamato (server) vengono eseguiti su macchine fisicamente distinte e, di conseguenza, si trovano in spazi di indirizzamento della memoria differenti.

Nelle chiamate di procedura convnzionali (locali), i parametri possono essere passati attraverso due logiche principali:

- **Per valore (call-by-value)**

Viene passata solo una copia del dato, eventuali modifiche effettuate dalla procedura chiamata non intaccano la variabile originale del chiamante.

- **Per riferimento (call-by-reference)**

Viene passato l'indirizzo di memoria (un puntatore) in cui si trova il dato.

In questo caso, la procedura lavora direttamente sull'oggetto originale e le sue modifiche sono visibili al chiamante.

In un sistema basato su RPC, **l'approccio del passaggio per riferimento è problematico e inapplicabile direttamente.**

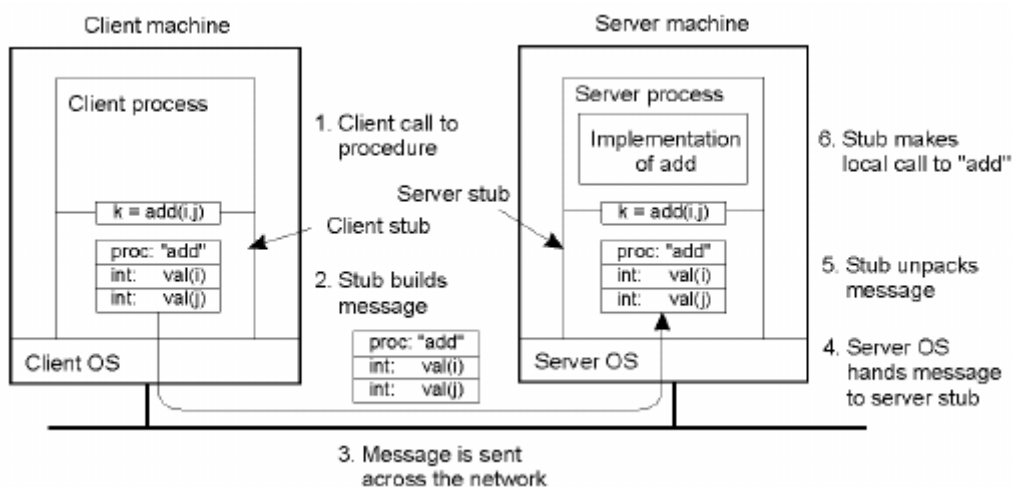
Gli indirizzi di memoria hanno infatti una valenza strettamente locale: inviare un puntatore della RAM del client al server non ha alcun senso, in quanto quell'indirizzo non corrisponderà alla stessa risorsa sulla macchina di destinazione.

19.4.1 Meccanismo call-by-copy/restore

Per aggirare l'ostacolo dell'approccio per passaggio di riferimento, si utilizza un meccanismo chiamato **call-by-copy/restore**, questo approccio si divide in 4 fasi:

1. Al momento della chiamata, il client copia il valore corrente della variabile (nello stack) e lo spedisce al server attraverso la rete.
2. Il server riceve i dati e la procedura remota esegue le sue operazioni lavorando sulla copia appena ricevuta.
3. Terminata l'elaborazione, il dato modificato dal server viene impacchettato e inviato nuovamente indietro nella risposta.
4. Il client riceve la risposta e **sovrascrive la sua variabile originale locale** con la versione aggiornata restituita dal server.

Grazie a questa sovrascrittura finale, l'effetto pratico sul client risulta totalmente analogo a quello di una chiamata per riferimento (call-by-reference), ma senza mai aver scambiato reali indirizzi di memoria.



19.5 Passaggio dei dati

Poichè le strutture dati complesse non possono viaggiare "così" come sono" su un cavo di rete, gli stub devono trasformarle.

Per fare ciò si effettuano 2 passaggi.

19.5.1 Serializzazione/Deserializzazione

È la conversione delle strutture dai presenti nella memoria RAM in un formato (come testo o sequenza di byte binari) adatto alla trasmissione sulla rete.

L'operazione inversa per ricostruire il dato in memoria si chiama **Deserializzazione**.

19.5.2 Marshalling / Unmarshalling

È il processo di impacchettamento vero e proprio dei dati serializzati all'interno del messaggio di rete.

L'**unmarshalling** è l'operazione inversa di estrazione e scompattamento.

Affinchè il client e server si capiscano senza errori durante queste operazioni di traduzione, il formato e la codifica dei dati devono essere rigorosamente condivisi tra i due stub.

Questo accordo viene tipicamente stabilito utilizzando dei linguaggi di definizione delle interfacce (IDL), come ad esempio i file `ProtoBuf` usati nella moderna implementazione `gRPC`.

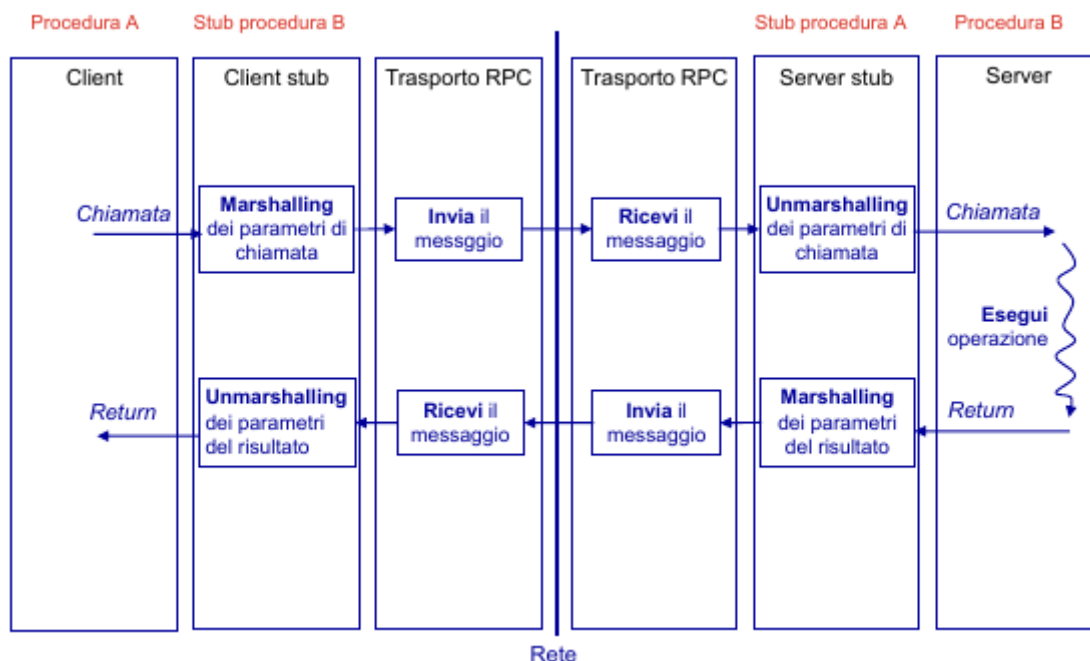


Figura 19.1: Esecuzione di una RPC

Capitolo 20

gRPC

20.1 Definizione

Sviluppato originariamente da Google nel 2015, è un framework RPC open source progettato per garantire **prestazioni elevatissime**.

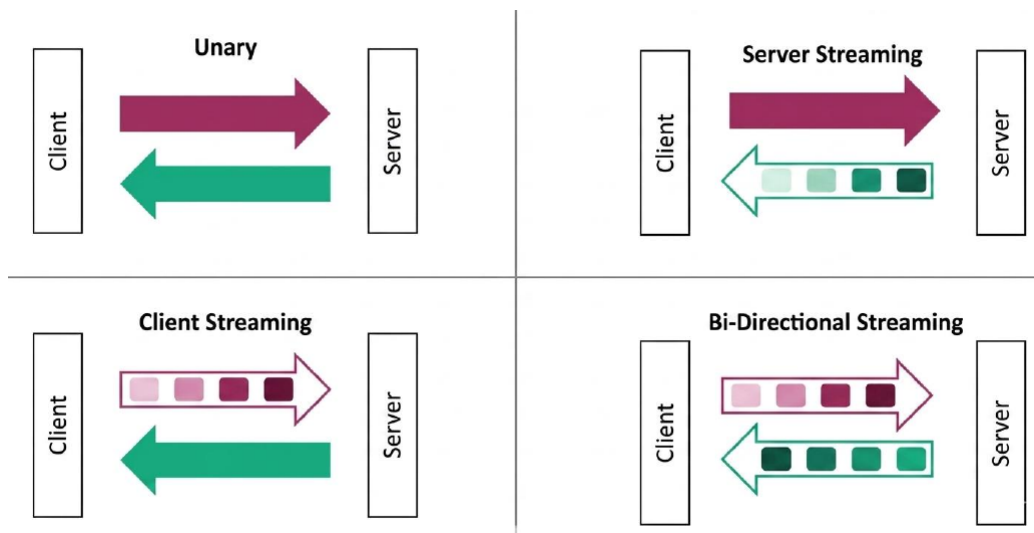
A differenza di protocolli più lenti e pesanti

20.2 Funzionalità di gRPC

20.2.1 Modelli di comunicazione

: Permette quattro **modelli di comunicazione**:

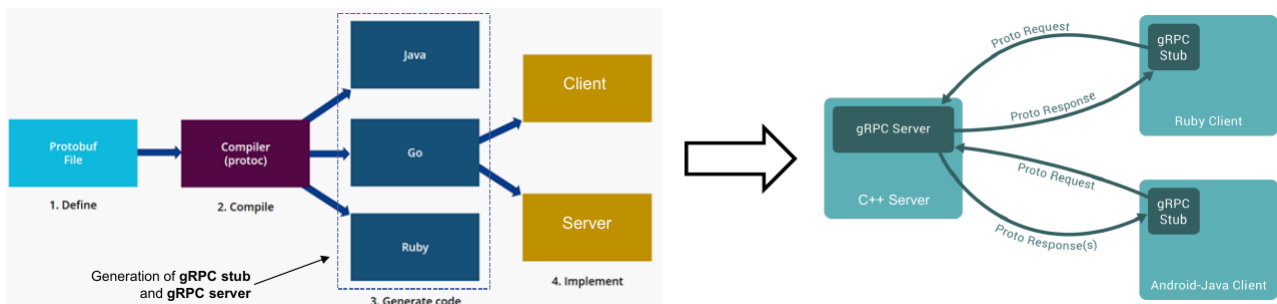
1. **Unary RPC**: Classico schema richiesta-risposta.
2. **Server Streaming**: Il client invia una richiesta e il server risponde con un flusso continuo di messaggi.
3. **Client Streaming**: Il client invia un flusso di messaggi e il server risponde una volta solo alla fine.
4. **Bidirectional Streaming**: Entrambi inviano flussi di messaggi simultaneamente.



20.2.2 File .proto

Con gRPC non bisogna scrivere manualmente il codice per marshalling e unmarshalling.

1. Si scrive un **file .proto**: definisce i messaggi (strutture dati) e i servizi (funzioni chiamabili remotamente).
2. Il codice viene compilato per mezzo del **compilatore gRPC**



Esempio di file .proto

Si vuole poter o chiedere lo stato attuale (Unary RPC) oppure ricevere aggiornamenti costanti (Streaming RPC)

```
1 // Specifica la versione della sintassi (proto3 e' lo standard attuale)
2 syntax = "proto3";
3
4 // Definizione dei messaggi: o i dati specificati dall'utente (SensorRequest) o
5 // creati dal sensore (EnvironmentData)
6 // I numeri (1, 2, 3) sono i "Tag", ovvero identificatori binari delle variabili
7 // usati in fase di comunicazione
8 message SensorRequest {
9     int32 sensor_id = 1; // ID del sensore specifico
10 }
11
12 message EnvironmentData {
13     float temperature = 1; // Temperatura
14     float humidity = 2;    // Umidita' in percentuale
15     string status = 3;     // Es. "OTTIMALE", "ALLERTA"
16 }
17
18 // Definizione del servizio: le procedure (funzioni) disponibili
19 service SystemMonitor {
20     // 1. Unary RPC: Il client chiede i dati e riceve una sola risposta
21     rpc GetCurrentStatus(SensorRequest) returns (EnvironmentData);
22
23     // 2. Server Streaming: Il client chiede di monitorare e il server invia un
24     // flusso continuo di dati (es. ogni sec.)
25     rpc WatchEnvironment(SensorRequest) returns (stream EnvironmentData);
26 }
```

20.2.3 Timeout

La chiamata viene annullata automaticamente dopo un certo tempo.

20.2.4 Cancellazione della chiamata

Viene inviata una notifica al server, il quale può smettere di lavorare.

20.2.5 Autenticazione

Supporto nativo (es. SSL/TLS)

Capitolo 21

Java RMI

21.1 Definizione

È un middleware che consente a oggetti su una JVM di accedere a o invocare un oggetto in esecuzione su un'altra JVM.

È un implementazione di RPC che **permette la comunicazione remota tra programmi** Java trasparente al programmatore.

Viene fornito dal pacchetto **java.rmi**

21.2 Architettura di un'applicazione RMI

21.2.1 Procedura

Vengono scritti due programmi: un **programma client** e un **programma server**.

- Nel programma server viene creato un **oggetto remoto**.
- Un **riferimento** a tale oggetto viene messo a disposizione del client.
- Il programma client richiede **l'accesso all'oggetto remoto** e cerca di **invocare i suoi metodi**

21.2.2 Problematiche fondamentali

- Come passare i parametri a un oggetto?
- Come passare il riferimento remoto relativo a un oggetto?
- Nel caso di riferimento locale, come passare lo stato dell'oggetto?
- Nel caso di riferimento locale, come ricostruire la struttura dell'oggetto lato server?

21.3 Ricostruzione dell'oggetto - Java Runtime Enviroment

In Java il problema della ricostruzione della struttura dell'oggetto è risolvibile nativamente grazie al Java Runtime Enviroment.

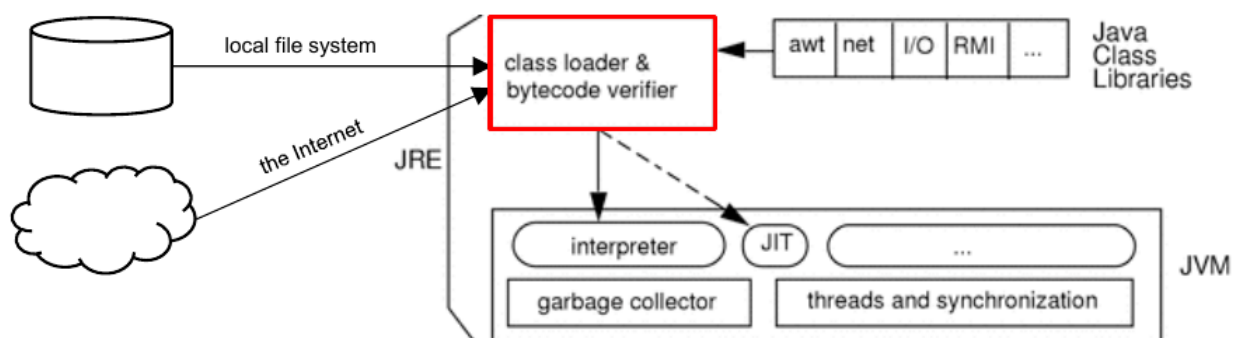
Il **class loader** ha il compito di caricare le classi (file .class, formato bytecode)

- I file .class possono essere trasferiti da librerie, file system locali o nodi remoti in rete.

- Il caricamento avviene dinamicamente quando
 - Viene eseguito il comando new, oppure
 - Un oggetto viene trasferito per mezzo di RMI (se il file .class non è presente localmente).

Il file .class include:

- **Struttura e metadati della classe.**
- **Tabella dei simboli.**
- **Bytecode dei metodi.**
- **Definizione dei campi.**



21.4 Passaggio dei parametri

21.4.1 Passaggio per valore

- Tipi primitivi
- Oggetti non remoti, devono implementare l'interfaccia `java.io.Serializable`.

21.4.2 Passaggio per riferimento

Per passare oggetti remoti si utilizza la classe `java.rmi.server.UnicastRemoteObject` e serve per implementare

Java.io.Serializable

Specifica che l'oggetto è un "contenitore di dati" e che può essere somantato in una sequenza di byte, spedito in rete e rimontato

- Passaggio per valore.
- Viene ricevuta una copia esatta dello **stato (variabili)** dell'oggetto originario.

java.rmi.Remote

Specifica che l'oggetto è remoto e i suoi metodi possono essere invocati da programmi eseguiti su altre macchine.

- Se un oggetto che implementa Remote viene passato come parametro in una chiamata remota, non viene inviato l'oggetto vero e proprio ma il suo **stub**.

Passaggio per riferimento.

- Lo stub è un oggetto includente il **riferimento**, ovvero:
 - Indirizzo IP del server.
 - Porca TCP del processo.
 - Identificativo univoco dell'oggetto.àà

La classe UnicastRemoteObject implementa Serializable per permettere la serializzazione dello stub.

21.5 Passaggio dello stato - Serializzazione dell'oggetto

La serializzazione rappresenta lo stato di un oggetto come **stream di byte**, è essenziale per poter memorizzare e ricostruire lo **stato** degli oggetti.

- Per definire oggetti persistenti.
- Per trasferire oggetti via rete.

Per mezzo della serializzazione devono essere passate solo le informazioni essenziali sullo stato, la descrizione della classe (struttura) è caricata a parte (file .class).

- La serializzazione usa il metodo `writeObject(Object obj)` della classe `ObjectOutputStream`
 - Attraversa ricorsivamente tutti i riferimenti ad altri oggetti contenuti in `obj` per costruire una rappresentazione completa del **grafo degli oggetti**.
 - Vengono serializzati tutti gli oggetti del grafo (devono implementare `Serializable`).

Il processo inverso (deserializzazione) usa il metodo `readObject()`.

Esempio (oggetto persistente)

```

1 // serializzazione di stringa e data odierna su un file
2 FileOutputStream f = new FileOutputStream("tmp"); // creazione canale di
   comunicazione per scrittura su file
3 ObjectOutputStream s1 = new ObjectOutputStream(f); // creazione oggetto per
   serializz.
4 s1.writeObject("Today"); // serializzazione stringa
5 s1.writeObject(new Date()); // serializzazione oggetto data
6 s1.flush(); // passaggio dei dati serializzati da buffer a file (quando buffer
   non pieno)
7
8 // deserializzazione di stringa e data da file
9 FileInputStream in = new FileInputStream("tmp"); //creazione canale di
   comunicazione per lettura da file
10 ObjectInputStream s2 = new ObjectInputStream(in); // creazione oggetto per
   deserializzazione
11 String today = (String)s2.readObject(); // deserializzazione stringa
12 Date date = (Date)s2.readObject(); // deserializzazione oggetto data

```

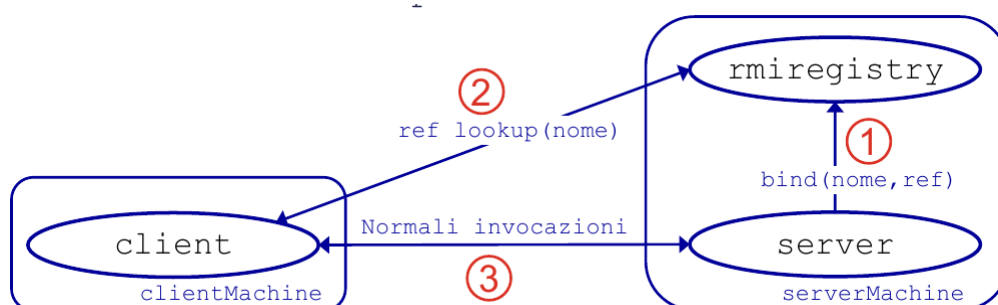
21.6 Passaggio dei riferimenti

RMI definisce un componente chiamato **RMI registry**

- Si ha un rmiregistry su ogni macchina che ospita oggetti remoti.
- Associa univocamente un nome logico assegnato all'oggetto al suo riferimento.
- RMI attiva un servizio di ascolto per quel servizio.

Viene utilizzata la classe `java.rmi.Naming` per accedere alle funzionalità dell'RMI registry

- Metodi fondamentali: `bind` e `lookup`.



21.6.1 La classe `java.rmi.Naming`

Parametri

- `name`: Nome logico dell'oggetto in formato URL.
- `obj`: Il riferimento all'oggetto remoto (solitamente un oggetto stub).

Metodi

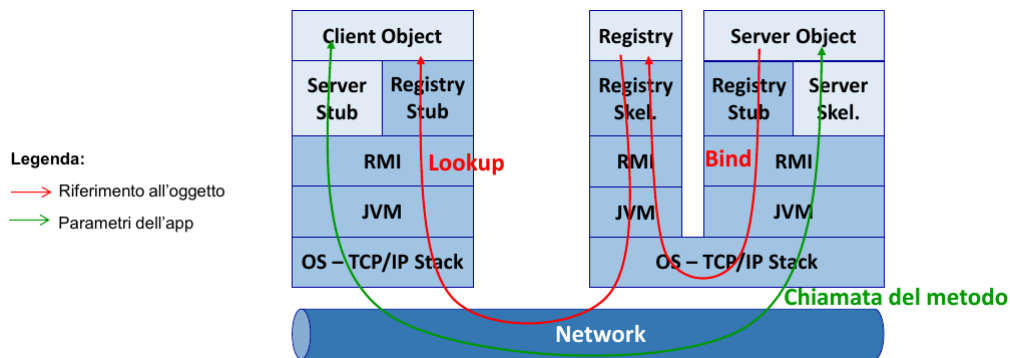
- `public static void bind(String name, Remote obj) throws AlreadyBoundException, MalformedURLException, RemoteException`
 - Collega il nome specificato all'oggetto remoto
- `public static void rebind(String name, Remote obj) throws RemoteException, MalformedURLException`
 - Collega (bind) il nome specificato all'oggetto remoto, cancellando i collegamenti esistenti
- `public static Remote lookup(String name) throws NotBoundException, MalformedURLException, RemoteException`
 - Restituisce un riferimento (stub) all'oggetto associato al nome specificato
- `public static String[] list(String name) throws RemoteException, MalformedURLException`
 - Restituisce i nomi (in formato URL) degli oggetti del registry
- `public static void unbind(String name) throws RemoteException, NotBoundException, MalformedURLException`
 - Distrugge il collegamento (bind) al nome specifico

Eccezioni

- `AlreadyBoundException`: Il nome è già collegato a un altro oggetto.
- `NotBoundException`: Il nome non è collegato ad alcun oggetto.
- `RemoteException`: Il registry non può essere contattato.
- `MalformedURLException`: Il nome non è un RUL formattato in modo appropriato.

21.6.2 Workflow in dettaglio

1. Il server pubblica il riferimento e il nome dell'oggetto remoto nel registry invocando il metodo `bind`.
2. Il client ottiene il Server Stub e il riferimento all'oggetto remoto invocando il metodo `lookup`.
3. Il client invoca il metodo sullo stub, il quale inoltra al server il riferimento dell'oggetto remoto e l'identificativo del metodo da eseguire, più i parametri del metodo.
4. Il registry ha stub e skeleton → È un oggetto remoto con cui bisogna comunicare.



21.7 Problematiche fondamentali dei sistemi distribuiti e Java RMI

In che modo le problematiche fondamentali dei sistemi distribuiti sono affrontate quando Java RMI è adottato per l'invocazione di metodi di oggetti remoti?

1. Identificazione della controparte (naming)
Sono necessari l'hostname e il nome dell'oggetto.
2. Accedere alla controparte (access point)
Accesso per mezzo del RMI registry
3. Comunicazione 1 (protocol)
Serializzazione degli oggetti locali o riferimento a oggetti remoti.
4. Comunicazione 2 (syntax and semantics)
Tipi di Java

21.8 Vantaggi e Svantaggi

21.8.1 Vantaggi (rispetto a implementazioni RPC generali)

- Supporto nativo a un design orientato agli oggetti.
- Integrazione totale e trasparente con il linguaggio Java.

21.8.2 Svantaggi (rispetto a implementazioni RPC generali)

- Legato indissolubilmente a Java.
- Overhead maggiore e prestazioni minori.
- Maggior complessità a livello di rete e problemi di sicurezza.

Capitolo 22

Javascript

22.1 Variabili

22.1.1 var

Sono variabili che hanno **function scope**, sono visibili in tutta la funzione che le contiene e non rispettano lo scope di blocco.

```
1 var x = 10; // qui x vale 10
2 {
3   let x = 2; // qui x vale 2
4   let y = 5; // qui y vale 5
5   var z = 8; // qui z vale 8
6 }
7
8 // qui x vale 10
9 // y NON è accessibile qui
10 // z può essere usata qui (var ignora il blocco)
```

Si usano per le variabili globali.

22.1.2 let

Sono variabili con **block scope**, esistono solo nel blocco { . . . } in cui è dichiarata.

22.1.3 const

Sono variabili con **block scope**, esistono solo nel blocco { . . . } in cui è dichiarata.

Non possono essere riassegnate dopo l'inizializzazione.

Sono contenitori costanti, ma i valori contenuti possono cambiare:

- Le variabili semplici sono modificabili.

```
1 var x = 10;
2 // Here x is 10
3 {
4   const x = 2; // Here x is 2
5   x = 4;       // ERROR!
6 }
```

```
7 // Here x is 10
```

- I valori delle variabili composte possono cambiare

```
1 const ages = [15, 35, 28, 18];  
2 // Here ages is 15, 35, 28, 18  
3  
4 for (let i = 0; i < ages.length; i++) {  
5   ages[i] = ages[i] + 1;  
6 }  
7 // Here ages is 16,36,29,19  
8 ages[4] = 56;  
9 // Here ages is 16,36,29,19,56  
10 ages = [43, 26];
```

22.2 Controllo di flusso

22.2.1 Strutture standard

if/else, switch, for, while, do while identici a java

22.2.2 Iteratori specifici di JS

- **for of**: Itera sui valori (array, stringhe, ecc).
- **for in**: Itera sulla chiave di un oggetto.

```
1 const arr = [10, 20, 30];  
2  
3 // for classico  
4 for (let i = 0; i < arr.length; i++) {  
5   console.log(arr[i]);  
6 }  
7  
8 // itera sui valori  
9 for (const v of arr) {  
10  console.log(v); // 10, 20, 30  
11 }  
12  
13 // itera sugli indici/chiavi  
14 for (const k in arr) {  
15  console.log(k); // 0, 1, 2  
16 }
```

22.3 Funzioni

22.3.1 Sintassi di una funzione

Esistono 3 modi per dichiarare una funzione:

1. Metodologia classica

```
1 function f(x, y){  
2     return x * y;  
3 }
```

2. Assegnazione ad una variabile

```
1 var f = function(x,y){  
2     return x * y;  
3 }
```

3. Arrow function

```
1 const f = (x,y) => {return x*y;}
```

In questo caso la parola chiave **return** e le parentesi graffe possono essere omesse solo quando la funzione ha una sola istruzione, per questa ragione è buona abitudine usarle sempre

22.4 Stringhe

22.4.1 Definizione

Una stringa in JS è una sequenza di 0 o più caratteri racchiusi tra virgolette

```
1 let carName1 = "Panda"; // doppie virgolette  
2 let carName2 = 'Punto'; // virgolette singole
```

22.4.2 Lunghezza di una stringa

Per ottenere la lunghezza di una stringa, si usa la proprietà `length` integrata.

```
1 let sln = carName1.length; // 5
```

22.4.3 Concatenazione di stringhe

L'operatore `+` può essere usato per concatenare le stringhe.

```
1 let txt1 = "John";  
2 let txt2 = 'Doe';  
3 let txt3 = txt1 + " " + txt2; // John Doe
```

Sommando un numero a una stringa si ottiene una stringa

```
1 let txt = Ciao;  
2 txt += 35; // Ciao35
```

22.4.4 Metodi

- Il metodo **indexOf()** restituisce l'indice (la posizione) della prima occorrenza di un testo specifico in una stringa.

```
1 const str = "Please locate where 'locate' occurs!";  
2 let pos = str.indexOf("locate"); // 7
```

Restituisce -1 se il testo non viene trovato.

Accetta un secondo parametro che indica la posizione di partenza per la ricerca.

- Il metodo **lastIndexOf()** restituisce l'indice dell'**ultima** occorrenza di un testo specifico in una stringa

```
1 let pos = str.lastIndexOf("locate"); // 21
```

Restituisce -1 se il testo non viene trovato.

Accetta un secondo parametro che indica la posizione di partenza per la ricerca.

- Il metodo **startsWith()** [**endsWith()**] restituisce **true** se una stringa inizia [termina] con un valore specificato, altrimenti **false** (case sensitive).

```
1 const str = "Please locate where 'locate' occurs!";  
2 let pos = str.indexOf("locate");
```

Un secondo parametro opzionale indica una posizione da cui iniziare [a cui terminare]

```
1 text.startsWith("world", 6); // controlla dalla posizione 6, restituisce true  
2 text.endsWith("world", 11); // controlla i primi 11 caratteri, restituisce true
```

- Il metodo **slice()** estrae una porzione di una stringa e restituisce la parte estratta in una nuova stringa.

Accetta 1 o 2 parametri: la posizione iniziale e la posizione finale (esclusa) [opzionale]

```
1 const str = "Apple, Banana, Kiwi";  
2 let pos = str.slice(7, 13); // // Banana  
3 pos = str.slice(7); // // Banana, Kiwi
```


22.5 Metodi per manipolare Array

- **Aggiungere elementi**

Il modo più semplice per aggiungere un nuovo elemento a un array è usare il metodo **push()**

```
1 const fruits = ["Banana", "Orange", "Apple", "Mango"];
2 fruits.push("Lemon"); // aggiunge un nuovo elemento ("Lemon")
3 let x = fruits.push(); // restituisce length: il valore di x è 5
```

- **Rimuovere l'ultimo elemento**

Il metodo **pop()** rimuove l'ultimo elemento da un array

```
1 fruits.pop(); // rimuove l'ultimo elemento ("Lemon")
2 let y = fruits.pop(); // il valore di y è "Lemon"
```

- **Rimuovere il primo elemento**

Il metodo **shift()** rimuove il primo elemento dell'array e fa scorrere tutti gli altri elementi a un indice inferiore.

```
1 fruits.shift(); // rimuove il primo elemento ("Banana")
2 let z = fruits.shift(); // il valore di z è "Banana"
```

- **Aggiungere in testa**

Il metodo **unshift()** aggiunge il primo elemento dell'array e fa scorrere tutti gli altri elementi a un indice superiore

```
1 fruits.unshift("Banana"); // aggiunge "Banana" in prima posizione
```

- **Splice di un array**

Il metodo **splice()** può essere usato per inserire nuovi elementi in un array

```
1 const fruits = ["Banana", "Orange", "Apple", "Mango"];
2 fruits.splice(2, 0, "Lemon", "Kiwi");
3 // fruits = ["Banana", "Orange", "Lemon", "Kiwi", "Apple", "Mango"]
```

- Il primo parametro indica la posizione in cui i nuovi elementi devono essere aggiunti (inseriti).
- Il secondo parametro definisce quanti elementi devono essere rimossi.
- I parametri successioni definiscono i nuovi elementi da aggiungere

Restituisce un array con gli elementi rimossi

```
1 let removed = fruits.splice(2, 2, "Lemon", "Kiwi");
2 // fruits = ["Banana", "Orange", "Lemon", "Kiwi"]
3 // removed = ["Apple", "Mango"]
```

22.6 Classi

22.6.1 Definizione

Permettono di definire dei tipi di oggetti e creare istanze.

In JS tutti i valori sono oggetti, tranne i tipi primitivi.

Quando scriviamo un letterale stiamo creando l'istanza di una classe predefinita

```
1 // Utilizzo del literal di array (scelta consigliata)
2 const names = ["Anna", "Diana"];
3
4 // Equivalente utilizzando il costruttore Array
5 const names = new Array("Anna", "Diana");
```

22.6.2 Anatomia di una classe

Elementi di una classe

- **constructor**: Viene eseguito da new e inizializza i campi tramite this.
- **Metodi**: Si scrivono direttamente nel corpo della classe, senza la parola function.
- **Campi**: Non si dichiarano a priori, nascono assegnando this.x = ... nel constructor

Differenze da Java

- Niente tipi nelle firme
- Niente public/private
- Niente overloading: una sola firma per nome

Esempio di classe

```
1 class Car {
2   // Il costruttore viene chiamato automaticamente quando crei un nuovo oggetto
3   constructor(brand, year) {
4     this.brand = brand; // Assegna il valore all'istanza dell'oggetto
5     this.year = year;
6   }
7
8   // Metodo per calcolare l'età dell'auto
9   age(currentYear) {
10    return currentYear - this.year;
11  }
12
13  // Metodo per restituire una stringa descrittiva
14  toString() {
15    return `${this.brand} (${this.year})`;
16  }
17 }
18
19 // Esempio di utilizzo:
```

```

20 // 'new' crea una nuova istanza (oggetto) della classe Car
21 const c = new Car("Fiat", 2020);
22
23 console.log(c.toString()); // Output: "Fiat (2020)"
24 console.log(c.age(2026)); // Output: 6

```

22.6.3 Iterazioni su variabili multivalore

for/of

Considera tutti i valori di una struttura iterativa

```

1 const numbers = [1, 2, 3, 4, 5, 6];
2 let sum = 0; // Inizializziamo la variabile che conterrà la somma
3
4 // Il ciclo for...of scorre ogni elemento dell'array
5 for (let x of numbers) {
6     sum = sum + x; // Aggiunge il valore corrente (x) alla somma totale
7 }
8
9 // Dopo il ciclo, sum vale 21 (1+2+3+4+5+6 = 21)
10 console.log(sum);

```

for/in

Considera tutte le proprietà di un oggetto

```

1 const person = {fname:"John", lname:"Doe", age:25};
2 let str = "";
3
4 // Il ciclo for...in scorre le chiavi dell'oggetto
5 for (let x in person) {
6     // Concatena la chiave (x), il separatore, il valore (person[x]) e un tag HTML
7     str += x + ": " + person[x] + "<br>";
8 }

```

22.7 Programmazione funzionale

22.7.1 forEach

Il metodo **forEach()** invoca una funzione (una funzione di callback) una volta per ciascun elemento dell'array

```
1 const numbers = [45, 4, 9, 16, 25];
2 let txt = "";
3 numbers.forEach(myFunction);
4
5 function myFunction(value, index, array) { // index e array sono opzionali
6     txt += value + "<br>";
7 }
```

La funzione accetta 3 argomenti:

- Il valore dell'elemento
- L'indice dell'elemento
- L'array stesso

22.7.2 map

Il metodo **map()** crea un nuovo array applicando una funzione a ciascun elemento dell'array.

- Non esegue la funzione sugli elementi privi di valore.
- Non modifica l'array originale

```
1 const numbers1 = [45, 4, 9, 16, 25];
2 const numbers2 = numbers1.map(myFunction);
3
4 function myFunction(value, index, array) { // moltiplica ogni valore per 2
5     return value * 2;
6 } // array risultante: [90, 8, 18, 32, 50]
```

La funzione accetta 3 argomenti:

- Il valore dell'elemento.
- L'indice dell'elemento.
- L'array stesso.

22.7.3 filter

Il metodo **filter()** crea un nuovo array contenente gli elementi che superano un test

- Non esegue la funzione sugli elementi senza valore.
- Non modifica l'array originale.

```
1 const numbers = [45, 4, 9, 16, 25];
2 const over18 = numbers.filter(myFunction);
3
4 function myFunction(value, index, array) {
```

```

5 // crea un nuovo array con gli elementi maggiori di 18
6 return value > 18;
7 } // array risultante: [45, 25]

```

La funzione accetta 3 argomenti:

- Il valore dell'elemento.
- L'indice dell'elemento.
- L'array stesso.

22.7.4 reduce

Il metodo **reduce()** esegue una funzione su ogni elemento dell'array per produrre un singolo valore.

- Lavora da sinistra a destra sull'array
- Non modifica l'array originale

```

1 const numbers = [45, 4, 9, 16, 25];
2 let sum = numbers.reduce(myFunction);
3
4 function myFunction(total, value, index, array) { // somma tutti i numeri di un array
5   return total + value;
6 } // valore risultante: 99

```

La funzione accetta 4 argomenti:

- L'accumulatore (il valore iniziale o il valore restituito in precedenza)
- Il valore corrente
- L'indice corrente
- L'array stesso

22.8 Visualizzare i dati

JS non ha funzioni native di stampa, per "mostrare" qualcosa si scrive nel DOM, in una finestra di dialogo o nella console

22.8.1 Scrivere nel DOM (modo principale)

- `element.textContent`: Inserisce solo testo, sicuro contro XSS
- `element.innerHTML`: Interpreta markup HTML, usare solo con contenuto fidato

22.8.2 Console del browser (debug)

- `console.log()`: stampa nella DevTools console

22.8.3 Finestre di dialogo (uso limitato)

- `alert()`, `confirm()`, `prompt()`: Bloccano l'interfaccia, usare solo per i prototipi o casi eccezionali

22.9 Gestione eventi

Javascript collega una funzione (handler) all'evento, e quel codice viene eseguito quando l'evento si verifica

22.9.1 Cos'è un evento?

Click, input da tastiera, submit di un form, caricamento pagina, scroll, mouseover, ...

22.9.2 Come aggiungere un Listener

Si usa il metodo `element.addEventListener(event, handler, options)`

- **tipo di evento**
 - Stringa: "click", "mousedown", "keyup", ...
 - Senza prefisso "on": "click", non "onclick"
- **handler**
 - Funzione anonima oppure nome di funzione (senza parametri)
 - Riceve l'oggetto Event come argomento
- **(opzionale) - options**
 - {once: true}, {capture: true}

Esempio

```
1 <!DOCTYPE html>
2 <html>
3 <body>
4   <button id="saveBtn">Salva</button>
5   <script>
6     const btn = document.getElementById("saveBtn");
7
8     // Handler con funzione anonima
9     btn.addEventListener("click", function (e) {
10       console.log("Cliccato!", e.target);
11     });
12
13     // Handler nominato + options
14     function onSave(e) { /* .. */ }
15     btn.addEventListener("click", onSave, { once: true });
16   </script>
17 </body>
18 </html>
```

Capitolo 23

DHTML

23.1 Definizione

DHTML non è un linguaggio specifico, ma l'integrazione di HTML, CSS, JS e DOM per rendere una pagina interattiva e dinamica lato client.



Questa nuova struttura permette di aggiornare il contenuto senza ricaricare la pagina, reagire agli eventi utente, modificare stile e struttura del documento, costruire interfacce più ricche e reattive

23.2 Componenti principali

- **HTML**: Struttura dei contenuti
- **CSS**: Presentazione e stile
- **JS**: Comportamento e logica
- **DOM**: Accesso e manipolazione degli elementi della pagina

23.3 Struttura di una pagina HTML5

Una pagina HTML5 ben formata ha una struttura minima fissa.

Lo scheletro è composto da 4 elementi annidati.

Esempio pagina HTML5

```
1 <!DOCTYPE html>
2 <html lang="it">
3 <head>
4   <meta charset="UTF-8">
5   <title>Pagina di esempio</title>
6 </head>
7 <body>
8   <!-- contenuto, UI, input/output -->
9   <script src="app.js" defer></script>
10 </body>
11 </html>
```

23.3.1 Best practice

- Il codice JS in file esterni, incluso con defer: il browser scarica lo script in parallelo all'HTML e lo esegue dopo il parsing
- In alternativa type="module" per usare i moduli

Capitolo 24

AJAX

24.1 Limite di DHTML

Può modificare contenuti e stile nel browser, ma da solo non introduce un meccanismo strutturato di comunicazione asincrona con il server

24.2 Definizione

È un pattern utilizzato per creare applicazioni Web interattive



24.3 Componenti

- **HTML e CSS**: Struttura e presentazione
- **Document Object Model (DOM)**: Rappresentazione ad albero della pagina, navigabile e modificabile da codice per aggiornamenti dinamici
- **XML e XSLT**: Formato originario per scambio e trasformazione dei dati tra client e server
- **Javascript**: Il "collante" che orchestra eventi, DOM e richieste XHR

24.3.1 AJAX come estensione di DHTML

AJAX aggiunge richieste asincrone (XMLHttpRequest o Fetch) verso il server, riceve i dati (oggi tipicamente JSON) e aggiorna solo una parte della pagina senza ricaricarla

24.3.2 Cosa abilita?

- Aggiornamento parziale della pagina (solo i dati che cambiano)
- Interfacce reattive, vicine all'esperienza desktop (rich UI)
- Riduzione del traffico di rete e dei tempi di attesa

24.4 WEB 2.0

AJAX è la tecnologia abilitante del **Web 2.0**

24.4.1 I 6 principi del Web 2.0

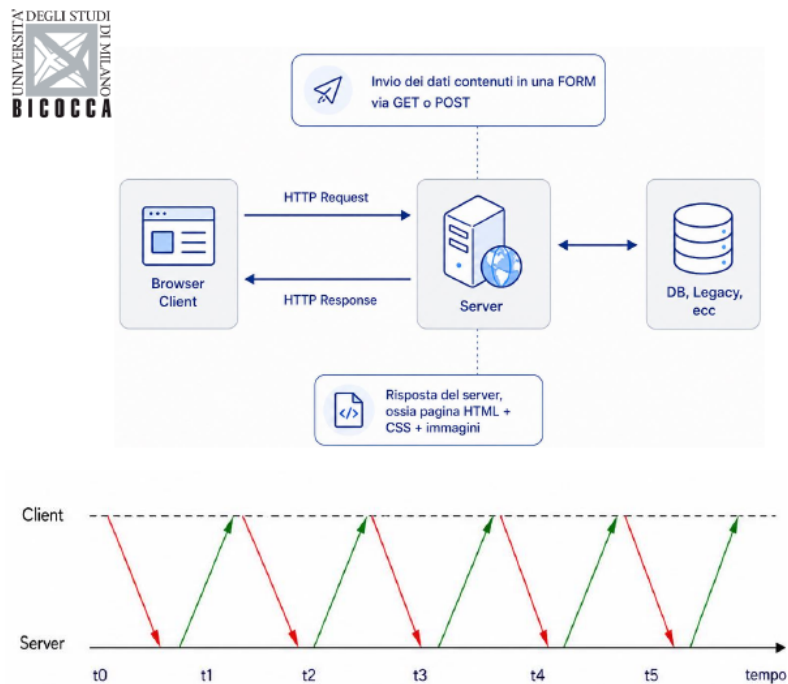
- **Il Web come piattaforma:** le applicazioni vivono nel browser, non sul desktop
- **Intelligenza collettiva:** Il valore cresce con il contributo degli utenti
- **Al di sopra del singolo dispositivo:** Servizi accessibili da qualsiasi client
- **I dati sono il nuovo "Intel Inside":** Il vantaggio competitivo sta nei dati gestiti
- **Esperienze utente ricche:** Interfacce reattive grazie ad AJAX

24.5 Utilizzi di AJAX

- **Validazione in tempo reale dei form**
- **Auto-completamento**
- **Operazioni master-detail**
- **Aggiornamento incrementale e live update**

24.6 Architettura Web con AJAX

24.6.1 Architettura Web Classica



- **Ciclo richiesta/risposta**

Il browser invia una richiesta HTTP.

Il server interroga il database e restituisce un'intera pagina HTML con CSS e immagini.

- **Andamento nel tempo**

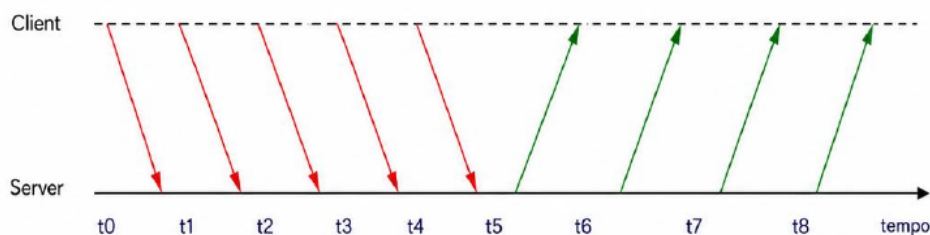
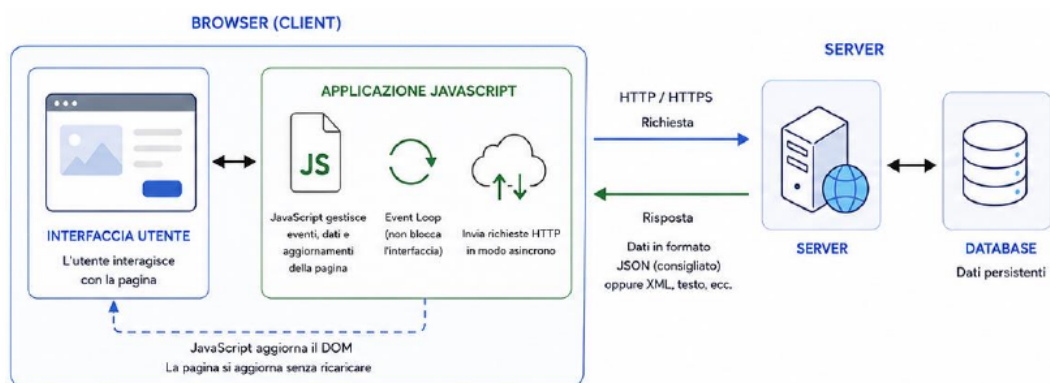
Ogni interazione produce un ciclo completo Request → Response.

Il client resta bloccato in attesa: l'utente non può fare altro finché la pagina non si è ricaricata.

- **Risultato**

Esperienza utente discontinua e traffico di rete sproporzionato rispetto all'informazione effettivamente nuova.

24.6.2 Architettura Web con AJAX



- **AJAX come intermediario**

Tra l'interfaccia utente e il server si inserisce un motore JS: l'UI dialoga con l'engine in locale (HTML + CSS), l'engine parla col server in HTTP scambiando solo dati (XML o JSON)

- **Asincrono**

Più richieste possono essere inviate senza attendere le risposte.

L'utente può continuare ad interagire mentre le risposte arrivano quando sono pronte e solo i dati necessari vengono aggiornati nella pagina.

24.7 Trasmissione dei dati

Lo scambio di dati tra client e server è il punto critico delle applicazioni web interattive: dimensioni del payload, parsing e compatibilità influenzando direttamente le presentazioni percepite.

24.8 Confronto modelli di esecuzione

Tre paradigmi differenti rispetto a chi guida il flusso di controllo e quando il programma termina

24.8.1 Sequenziale

Flusso

1. Il programma decide cosa fare e quando farlo
2. Esegue input → elabora → output → termina

Caratteristiche

- Terminazione esplicita
- Nessuna interazione durante l'esecuzione

Esempi

- Compilatore
- Script di build
- job batch

24.8.2 Interattivo

Flusso

1. Il programma chiede input, attende, elabora
2. L'utente risponde a richieste predefinite

Caratteristiche

- Controllo guidato dal programma
- Sequenza prevedibile di passi

Esempi

- CLI a prompt
- wizard
- REPL

24.8.3 Reattivo (event-driven)

Flusso

1. Il programma reagisce a eventi esterni
2. Resta in attesa, non termina

Caratteristiche

- Controllo guidato dall'ambiente
- Asincrono, non deterministico

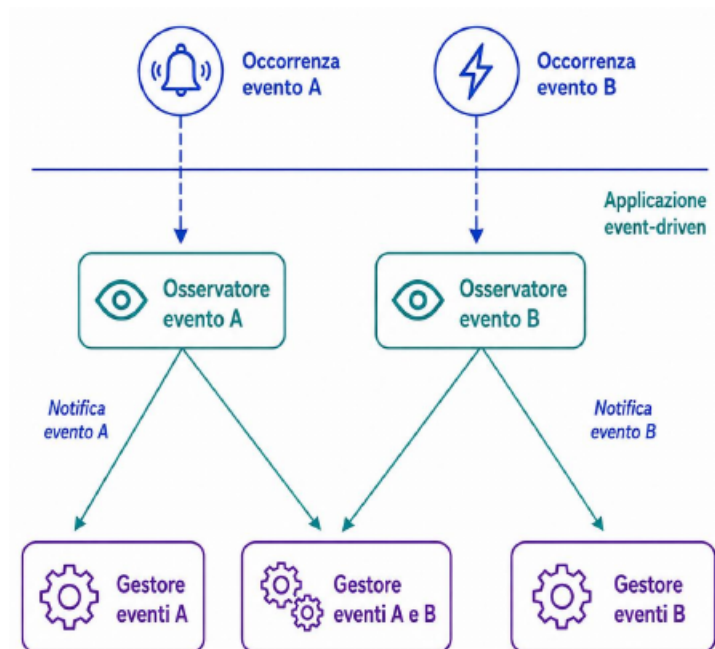
Esempi

- GUI
- Pagine web
- Server
- Embedded

24.9 Sistemi reattivi

24.9.1 Definizione

Un sistema reattivo mantiene un'interazione continua con il suo ambiente, rispondendo a stimoli esterni con tempi e modalità dettati dall'ambiente stesso.



24.9.2 Caratteristiche

- **Non-terminazione:** Il sistema resta vivo, in attesa del prossimo evento
- **Asincronia:** Gli eventi arrivano in tempi non prevedibili a priori
- **Concorrenza:** Più sorgenti di eventi attive simultaneamente

Non è possibile identificare staticamente un flusso di controllo unitario, il programma principale si limita a inizializzare l'applicazione, istanziando gli osservatori e associandovi gli opportuni handler. Questa associazione può essere dinamica.

24.9.3 Modello di esecuzione

- **Single-threaded**(in JS): Un solo handler alla volta
- **Run-to-completion:** Ogni handler termina prima del successivo
- **Conseguenza:** Handler lunghi bloccano l'intera UI

24.10 Formati e protocolli AJAX

- **JSON**: standard de-facto, compatto, parsing nativo nel browser
- **REST + JSON**: pattern dominante per le API web moderne
- **GrapQL**: alternativa che permette al client di richiedere solo i campi che gli servono
- **XML**: formato originario di AJAX, oggi raro nelle web app ma diffuso in contesti enterprise
- **SOAP e XML-RPC**: storici, sopravvivono in ambito legacy

24.11 Criticità di AJAX

24.11.1 Navigazione e bookmark

- Il tasto "back" e i preferiti registrano cambi di URL: con AJAX la pagina cambia contenuto senza cambiare URL, e l'utente perde il riferimento allo stato
- Mitigato dalla History API, permette di aggiornare URL e cronologia senza ricaricare

24.11.2 Indicizzazione e SEO

- I contenuti caricati via AJAX possono non essere indicizzati, storicamente i crawler non eseguivano JS, oggi lo fanno ma con limiti
- Soluzioni moderne: Server-Side Rendering e idratazione

24.11.3 Complessità del client

- La logica si sposta sul client, il bundle JS cresce, allungando il tempo di first load
- Debug più difficile, la stessa logica è distribuita tra client e server

24.11.4 Pressione sul server

- AJAX rende facile fare richieste piccole e frequenti, l'effetto cumulativo può pesare più del singolo full reload

Capitolo 25

XMLHttpRequest - Interazione server AJAX

25.1 Cos'è?

È un API JS fornita da tutti i browser moderni per inviare richieste HTTP e ricevere risposte senza ricaricare la pagine.

25.2 Cosa offre?

- **Modello asincrono**: La richiesta non blocca la UI, alla risposta scatta un evento.
- **Ciclo di vita esposto**: La proprietà `readyState` indica a che punto è la richiesta.
- **API a callback**: Si registra un handler su `onreadystatechange` che reagisce ai cambiamenti di stato.

25.3 Metodi principali

Metodo	Cosa fa
<code>new XMLHttpRequest()</code>	Crea l'oggetto richiesta
<code>setRequestHeader(k, v)</code>	Aggiunge un header HTTP (es. Content-Type)
<code>send(body)</code>	Invia la richiesta body = null per GET
<code>abort()</code>	Interrompe una richiesta in corso
<code>getResponseHeader(name)</code>	Legge un header HTTP della risposta

25.3.1 Schema d'uso

1. Creare l'oggetto
2. Registrare `onreadystatechange`
3. `open()` con i parametri
4. Eventuali `setRequestHeader`
5. `send()`

`setRequestHeader` serve quando si invia un POST con dati form: imposta header HTTP come Content-Type prima di `send()`

25.4 Proprietà di interesse

Proprietà	Significato
readyState	Stato del ciclo di vita (0-4)
status	Codice HTTP della risposta
responseXML	Corpo della risposta parsato come DOM XML
onreadystatechange	Handler chiamato a ogni cambio di stato
statusText	Descrizione testuale dello status

25.4.1 Sincorno o asincrono

Il terzo parametro di `open(method, url, async)` sceglie la modalità di esecuzione

async = true (raccomandato)

- Il browser invia la richiesta e continua a eseguire il resto dello script: la UI resta attiva
- La risposta arriverà più tardi: bisogna registrare un handler `onreadystatechange` prima di chiamare `send()`

async = false (sconsigliato)

- `send()` blocca il thread JS fino alla risposta: la pagina si congela
- Deprecato sul main thread, tutti i browser mostrano un warning, da evitare in produzione

25.5 Ciclo di vita

Il browser, tramite l'oggetto XMLHttpRequest, attraversa diverse fasi durante la comunicazione con il server.

La proprietà readyState indica in quale fase ci troviamo.



Il flag che ci interessa è `readyState == 4` (richiesta completata).

Accoppiato a `status == 200` indica una risposta utilizzabile.

L'evento `onreadystatechange` viene scatenato a ogni transizione: il nostro handler tipicamente filtra solo lo stato 4

25.5.1 Cos'è readyState?

È una proprietà dell'oggetto XMLHttpRequest che rappresenta lo stato interno della richiesta HTTP.

Viene aggiornata automaticamente dal browser e il suo valore cambia ogni volta che la richiesta avanza in una nuova fase.

25.5.2 L'evento readystatechange

Ogni volta che readyState cambia, il browser scatena l'evento readystatechange.

Possiamo intercettarlo per eseguire del codice al momento giusto

25.5.3 Risposta del server

Proprietà	Descrizione
responseText	Restituisce i dati della risposta come stringa
responseXML	Restituisce i dati della risposta come XML

Proprietà responseText

- Se la risposta dal server non è XML, usare la proprietà responseText
- La proprietà responseText restituisce la risposta come stringa

Proprietà responseXML

- Se la risposta dal server è XML e si vuole analizzare come oggetto XML, usare la proprietà responseXML

25.6 Richieste con fetch() - alternativa moderna e più semplice

fetch() è l'API moderna per richieste HTTP.

Restituisce una Promise, niente readyState, niente onreadystatechange, è disponibile in tutti i browser moderni e in Node.js dalla 18

XMLHttpRequest

```
1 const xhr = new XMLHttpRequest();
2 xhr.onreadystatechange = function() {
3     if (xhr.readyState === 4 && xhr.status === 200) {
4         const data = JSON.parse(xhr.responseText);
5         use(data);
6     }
7 };
8 xhr.open("GET", "/api/data", true);
9 xhr.send();
```

fetch + async/await

```
1 async function load() {
2     try {
3         const res = await fetch("/api/data");
4         if (!res.ok) throw new Error(res.status);
5         const data = await res.json();
6         use(data);
7     } catch (err) {
8         console.error(err);
9     }
10 }
```